Linux Systems and Performance

PART 2: CPU Performance and Scheduling

Week 5: Scheduling Algorithms in Linux — From Cache Coherence to EEVDF

Youngjae Kim, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr/

E-mail: youkim@sogang.ac.kr



Roadmap of This Week

PART 1

How do cores share data?



- Coherence problem
- Snooping, DSU
- MESI on big.LITTLE

PART 2

How do schedulers decide, in theory?



- Turnaround vs. response
- MLFQ: learn priority
- Lottery, stride: share

PART 3

How does each class pick the next task?



- Classes & their algorithms
- 3-1 Deadline: EDF + CBS
- 3-2 Real-time: FIFO / RR, PI
- 3-3 Fair: CFS → EEVDF

PART 4

How does it look on webOS?



- Video: DEADLINE sizing
- Audio: FIFO, PI
- UI: weight + slice
- Choosing a policy

This week

How cores keep shared data consistent (MESI), the scheduling theory behind Linux (MLFQ, lottery, stride), how each class picks the next task (EDF, CFS, EEVDF), and how the class algorithms apply to webOS threads.

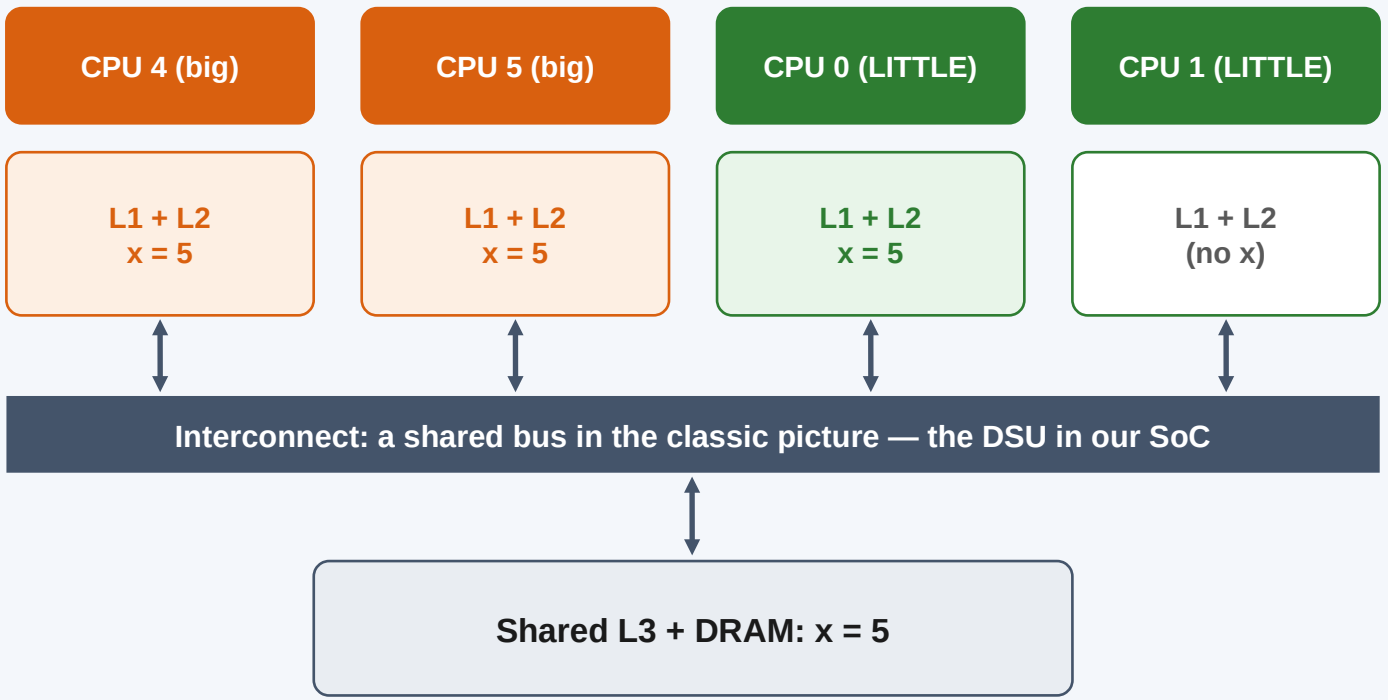
PART 2: CPU Performance and Scheduling

- **Week 5**

- **Cache Coherence: MESI on big.LITTLE**
  - How do cores keep their private caches consistent?
- **Scheduling Fundamentals: MLFQ and Proportional Share**
  - Metrics → MLFQ (learn priority) → lottery & stride (share) → the bridge to Linux
- **Scheduling Classes and Their Algorithms**
  - 3-1 Deadline (EDF, CBS) → 3-2 Real-time (FIFO / RR) → 3-3 Fair (CFS, EEVDF)
- **Scheduling Classes on webOS**
  - Video (DEADLINE) → audio (FIFO, PI) → UI (EEVDF) → choosing a policy

Shared-Memory Multiprocessors: Private Caches, One Memory

Shared memory on our big.LITTLE SoC (UMA)



x now lives in four places: L3/DRAM and three cores' private caches — big and LITTLE alike.

Programming view

- Any core can load / store any address
- Threads communicate implicitly through shared variables
- Caches are invisible to the program — hardware must keep copies consistent

Hardware view

- Private caches cut latency and bus traffic
- A bus is a broadcast medium → snooping (next slides)
- Our SoC: DSU snoop filter + coherent interconnect (later in this section)

Private caches make shared memory fast — and create multiple copies of the same data.

The Cache Coherence Problem, Step by Step

Write-through

① Both read X

big CPU 4	LITTLE CPU 0
X = -100	X = -100

L3 / DRAM: X = -100

both caches hold a copy

② CPU 0 writes X = 505

big CPU 4	LITTLE CPU 0
X = -100	X = 505

L3 / DRAM: X = 505

L3/DRAM updated (write-through)

③ CPU 4 reads X

big CPU 4	LITTLE CPU 0
X = -100 ×	X = 505

L3 / DRAM: X = 505

CPU 4 hits its stale copy

Write-back

① Both read X

big CPU 4	LITTLE CPU 0
X = -100	X = -100

L3 / DRAM: X = -100

both caches hold a copy

② CPU 0 writes X = 505

big CPU 4	LITTLE CPU 0
X = -100	X = 505 [dirty]

L3 / DRAM: X = -100

L3/DRAM is now stale too

③ CPU 4 reads X

big CPU 4	LITTLE CPU 0
X = -100 ×	X = 505 [dirty]

L3 / DRAM: X = -100 ×

CPU 4 hits stale; any core that misses gets stale L3/DRAM

A write policy keeps memory up to date at best — keeping the other caches correct needs a coherence protocol.

What Does "Coherent" Mean?

1. Write propagation

- A write by one core eventually becomes visible to all other cores
- No core may read its stale copy forever

2. Write serialization

- All writes to the same location are seen in the same order by every core
- If CPU 1 sees 1 then 2, CPU 5 must not see 2 then 1

Serialization example



CPU 5 reading 2 → 1 would violate coherence.

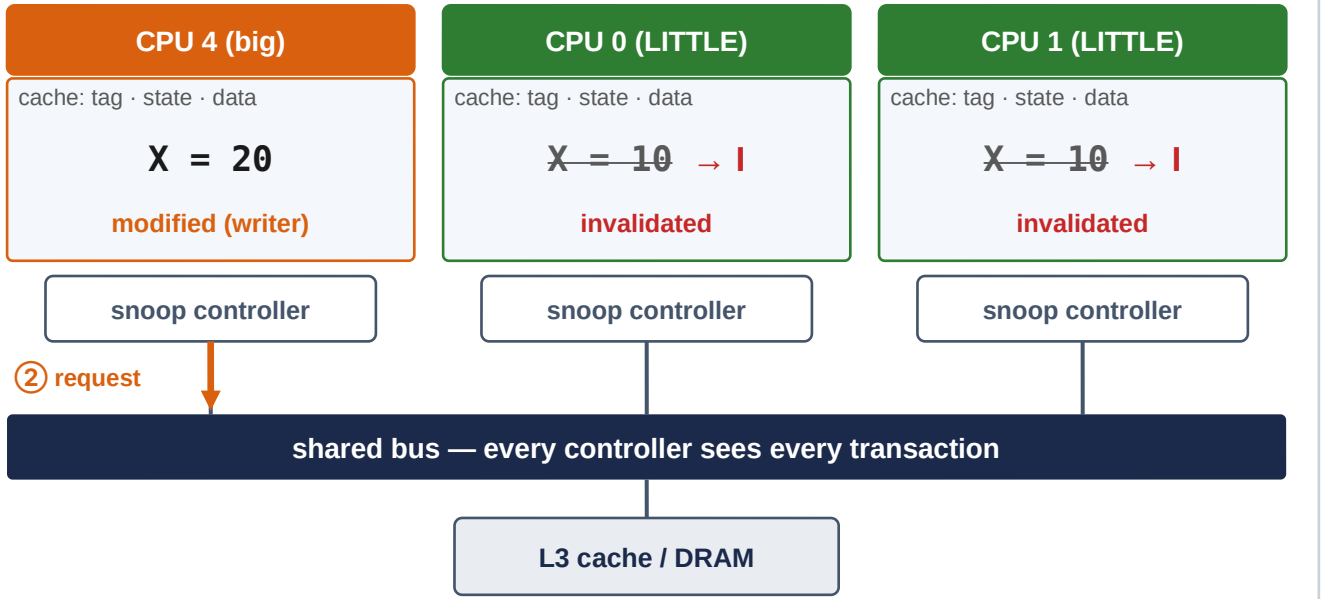
Coherence ≠ consistency

- **Coherence**: order of writes to **one** address. **Consistency**: order across **different** addresses.
- **Example (x = y = 0 at start):** CPU 4 writes x = 1, then y = 1. CPU 0 reads y, then x.
- If CPU 0 sees `y = 1 but x = 0`, coherence still holds: x alone goes 0 → 1 and y alone goes 0 → 1, and every core agrees on each history.
- Is consistency then broken? It depends on the **memory model**: under sequential consistency this result is forbidden → a violation; under a relaxed model (e.g., Arm) it is allowed → not a violation.

Coherence = every core agrees on the history of each single memory location.

Bus Snooping: Every Cache Controller Listens

Problem: X = 10 sits in three private caches. CPU 4 writes X = 20 → CPU 0 and CPU 1 still hold **stale 10** — CPU 0 would read the old value.



1. CPU 4 wants to write X
2. its cache controller puts a request on the bus
3. the snoop controllers of CPU 0 and CPU 1 see it
4. each checks its cache tags: do I hold X?
5. yes → mark that cache line Invalid
6. later, CPU 0 / CPU 1 read X → cache miss → fetch the new 20

Key point: CPU 4 never edits another core's cache. It **posts a request on the bus**, and every other cache controller **reacts on its own** — invalidate its copy, or supply data if it has the newest one.

Snooping: **every cache controller watches the shared bus** and reacts to other cores' reads and writes.

Bus Transactions by Example, and Snoop Filters

Transaction	Meaning	Other caches react	Example: CPU 4 and address X
`BusRd`	read miss	a cache with a dirty (modified) copy supplies the data; clean copies keep sharing	CPU 4 reads X; CPU 1 holds a modified X → CPU 1 supplies the newest value
`BusRdX / RFO`	read for ownership: fetch + get exclusive	invalidate every other copy	CPU 4 writes X but has no copy → fetch X and invalidate CPU 0, CPU 1
`BusUpgr`	upgrade (not update): write permission for a copy you already share	invalidate every other copy — never update it	CPU 4 already shares X → invalidate others, then write (next slide)
`Flush / write-back`	send modified data out	memory or the requester takes the data	CPU 4's modified X is requested by CPU 0, or evicted

BusRdX vs. BusUpgr: the only difference is whether the data must be fetched — worked example on the next slide.

Why broadcast does not scale

Good: everyone sees one order of requests → easy to keep coherent.

4 CPUs: one request → 3 snoops

64 CPUs: one request → up to 63 snoops

Cost: bus traffic · wasted tag lookups · power · bandwidth

Our SoC: the DSU snoop filter

Tracks which cores may hold each line. CPU 4 writes X:

CPU 0: may hold X | CPU 1: no X

→ snoop only CPU 0

fewer snoops than broadcasting. Details depend on the SoC and interconnect.

Link to scheduling

Renderer moves LITTLE CPU 0 → big CPU 4. Its data is still in CPU 0's cache → misses on CPU 4.

Coherence: the values are correct.

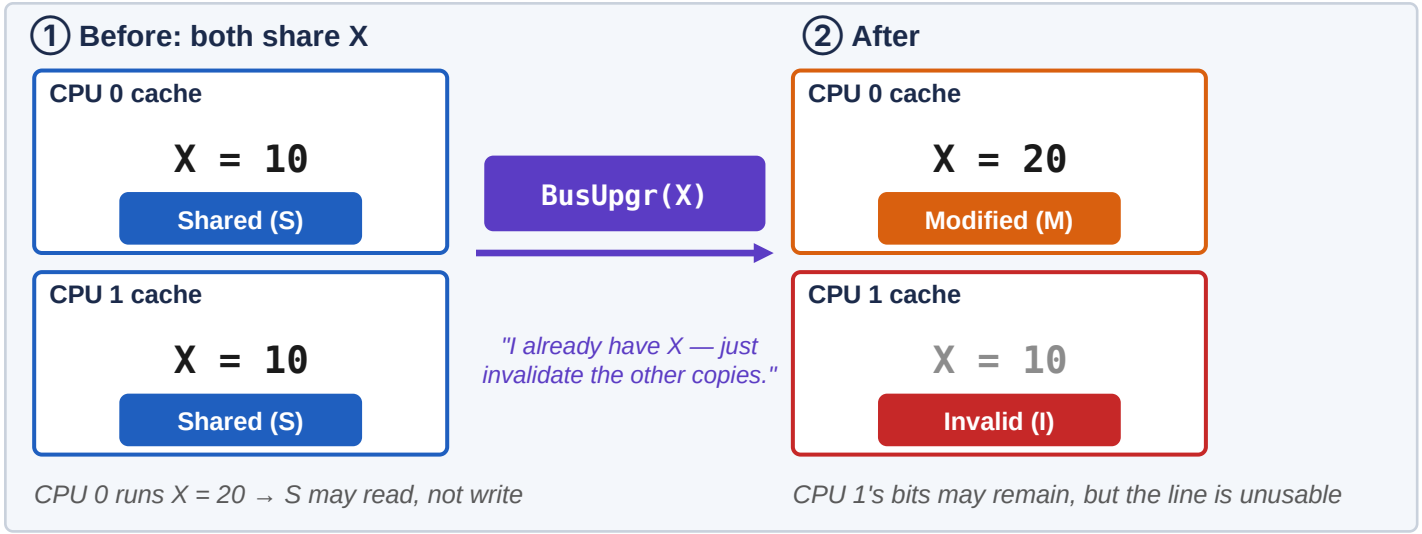
But: the misses cost time → the load balancer checks **cache-hot** before migrating.



Snoop to stay coherent, **filter the snoops to scale** — and remember: coherence keeps data correct, not cache-warm.

BusUpgr: Upgrade, Not Update

BusUpgr = Upgrade, not Update: get write permission for a line you already hold in **Shared (S) → Modified (M)** by **invalidating** the other copies.



1. CPU 0 issues BusUpgr(X)
2. CPU 1 marks its X line Invalid
3. CPU 0 now has write permission
4. CPU 0 writes X = 20 → Modified

CPU 1's X does NOT become 20. It is only invalidated; its next read misses and fetches 20 from CPU 0.

	BusUpgr	BusRdX
X already in my cache?	yes (Shared)	no
Fetch the data?	**no**	**yes**
Invalidate other copies?	**yes**	**yes**
Gets	write permission only	data + write permission

Two ways to keep copies coherent

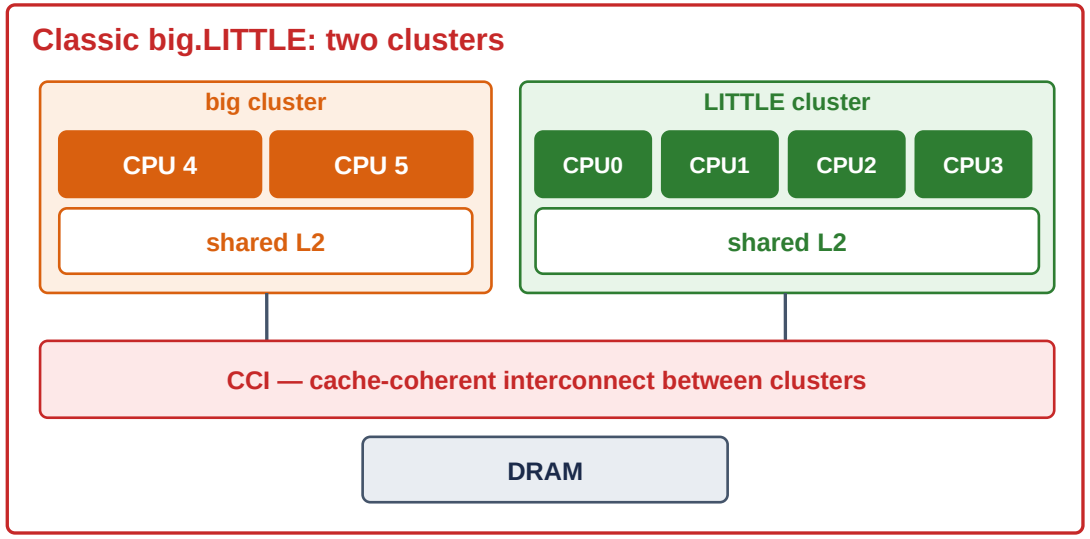
Write-invalidate (MESI, `BusUpgr`): on a write, other copies become Invalid.

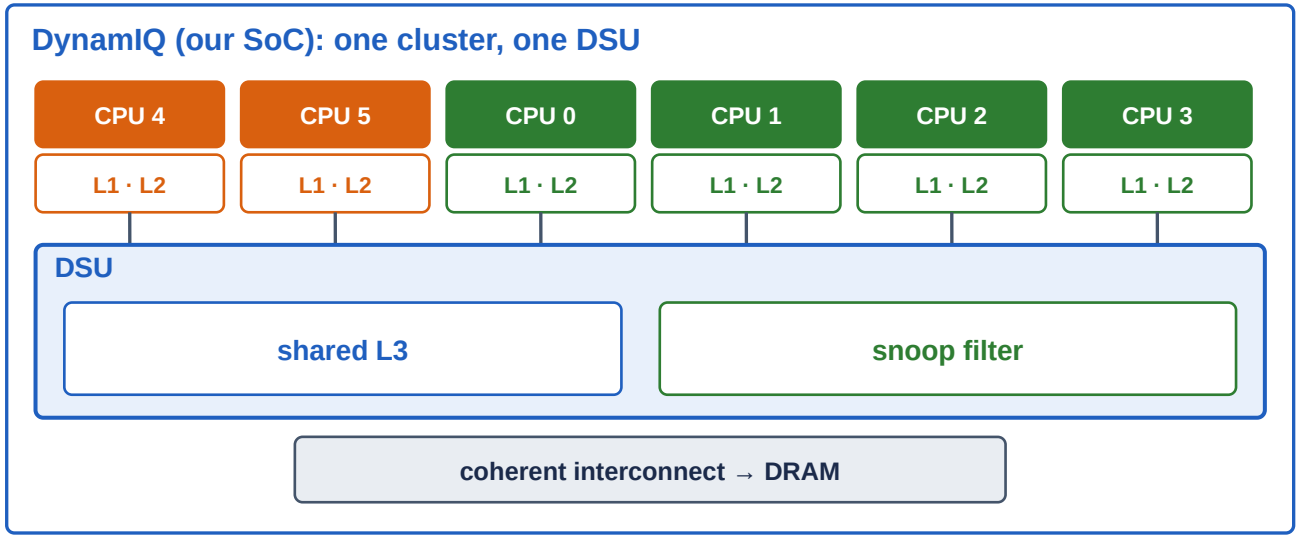
Write-update (some other protocols): the new value 20 is sent to the other caches and they are updated.

BusUpgr invalidates; it never updates.

BusRdX: **"I have no X — send it and make it mine."** BusUpgr: **"I have X — just make it mine."**

On Our SoC: Where Do big and LITTLE Meet?





A cache line moves between …

	Classic big.LITTLE	DynamIQ
big ↔ big	L2 of the same cluster	inside the DSU
LITTLE ↔ LITTLE	L2 of the same cluster	inside the DSU
big ↔ LITTLE	**across clusters via CCI**	**inside the DSU (L3 + snoop filter)**

Is coherence no longer needed?

No — the DSU is what keeps big and LITTLE coherent. The snoop filter tracks which cores may hold a line and snoops only those: **coherence done more efficiently, not removed.**

Private L1 / L2 remain, so reading another core's line still costs time.

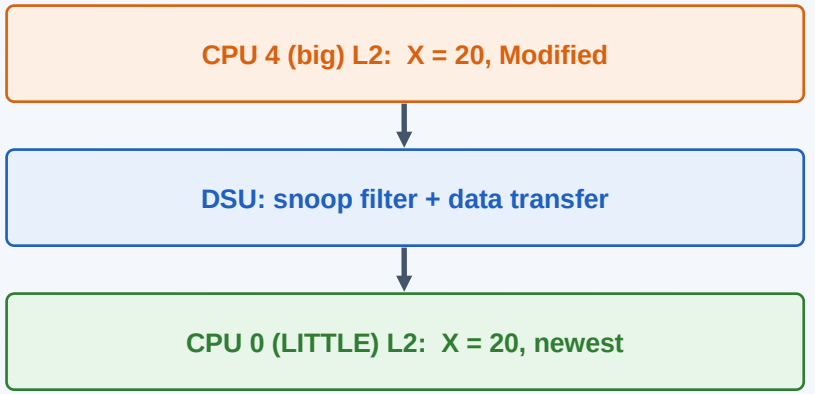
Classic: big and LITTLE meet **across clusters (CCI)**. DynamIQ: they meet **inside one DSU**, sharing L3 and the snoop filter.

Example: A Cache Line Travels from big to LITTLE

Renderer on big CPU 4 writes frame-buffer data X; a thread on LITTLE CPU 0 then reads X.

1	**CPU 4 writes**	`X = 20;`	CPU 4: X = 20, Modified · CPU 0: no X (Invalid). L3 / DRAM may still hold the old value.
2	**CPU 0 reads**	`int value = X;`	not in CPU 0's L1 / L2 → cache miss → read request to the DSU
3	**snoop filter checks**		the DSU sees that CPU 4 may hold X → snoops only CPU 4; its Modified copy must be supplied
4	**CPU 0 gets 20**		data goes CPU 4 L2 → DSU → CPU 0 L2 — no trip to DRAM needed

Path of the line



Whether the line also lands in L3 or goes core-to-core depends on the implementation and cache state.

Link to scheduling: migration CPU 4 → CPU 0

The renderer's data stays in CPU 4's L1 / L2. On CPU 0 it must be fetched again:

- cache misses on CPU 0
- snoop + transfer through the DSU
- refilling CPU 0's caches

Remember

1. DynamIQ puts different CPU types in one cluster that shares the DSU.
2. L3 + snoop filter deliver the newest data without going to DRAM.
3. Migration gets cheaper, **not free**: cache locality is still lost.

Inside the DSU, the newest copy goes **core → DSU → core** — cheaper than crossing clusters, yet migration still costs misses.

From Write-Invalidate to MESI: Giving Each Line a State

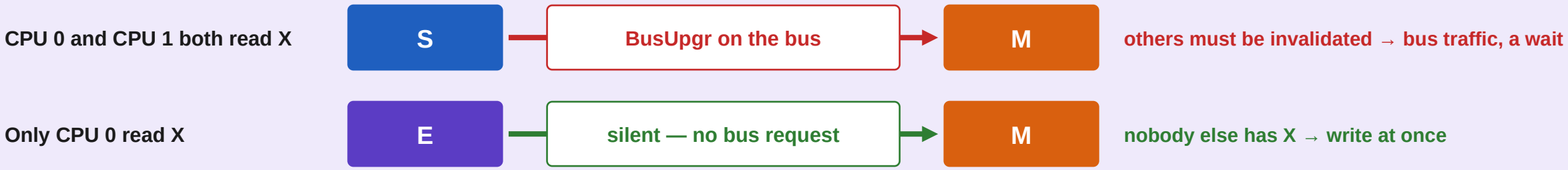
The write-invalidate snooping of the previous slides, written down as rules: every cache line carries one of four states — that is MESI.

What we called it on the previous slides	MESI state	Meaning
"the modified (dirty) copy" — CPU 4 wrote X = 20	**M Modified**	only copy, newer than memory
"a clean copy many cores share" — both read X = 10	**S Shared**	clean, other cores may have it too
"invalidated" — CPU 1 after BusUpgr	**I Invalid**	not usable; the next read misses
new: "I am the only one, and it is clean"	**E Exclusive**	only copy, same as memory → can write silently

The BusUpgr example, with state names

both read X	CPU 0: S	CPU 1: S
CPU 0 BusUpgr, writes	CPU 0: M	CPU 1: I
only CPU 0 read X	CPU 0: E	CPU 1: I
CPU 0 writes (silent)	CPU 0: M	CPU 1: I

The one new state, E: if only CPU 0 has read X, a write needs **no bus request at all**



MESI Protocol: Four States and Five Transitions

State	Meaning	Others have it?	Write without bus?
M	modified: only copy, newer than memory	no	yes
E	exclusive: only copy, same as memory	no	yes (→ M)
S	shared: others may have it too	maybe	no — invalidate first
I	invalid: no usable data	—	no — must fetch

E vs. S is the key difference

E: only mine → write at once. **S:** others may have it → invalidate them first.

Five transitions, with code

A first read — `int a = X;` — **I → E** — read miss, no one else has X

B another core reads — `CPU 0: read X` — **E → S · I → S** — both can read now

C write while shared — `X = 20;` — **S → M · S → I** — BusUpgr invalidates CPU 0

D write a private line — `X = 20;` — **E → M** — silent: no bus transaction

E another core reads a modified line — `CPU 0: read X` — **M → S · I → S** — CPU 4 supplies the data

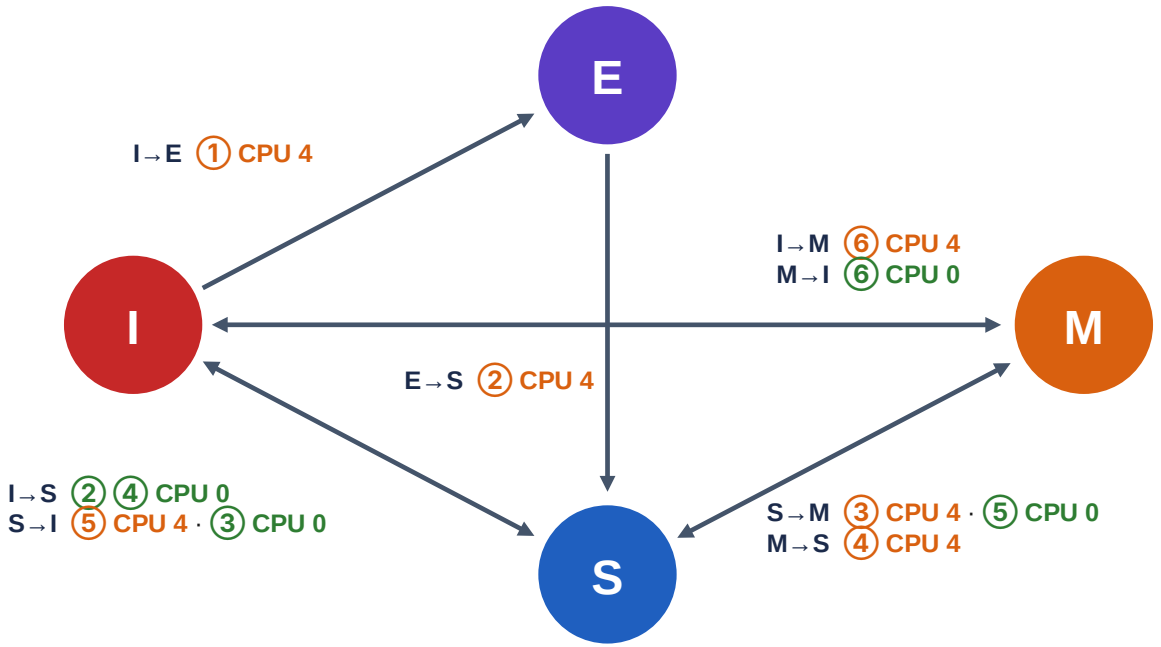
In B, C and E the second state is CPU 0's.

The rule that keeps caches coherent

Many cores may hold a line for **reading** at the same time — but once one core holds it for **writing (M)**, no other core may keep a valid copy.

MESI Walk-through: big CPU 4 and LITTLE CPU 0, One Line

The walk-through on the MESI diagram CPU 4 · CPU 0



Numbers = steps in the table; colour = which CPU's line moves.

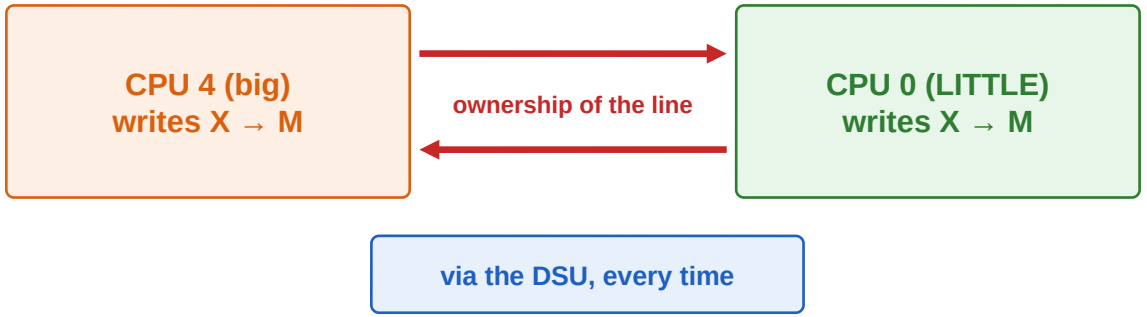
#	Action	CPU 4	CPU 0	Traffic
1	**CPU 4 reads X**	E	I	read from L3 / DRAM
2	**CPU 0 reads X**	S	S	read, CPU 4 says "shared"
3	**CPU 4 writes X**	M	I	BusUpgr
4	**CPU 0 reads X**	S	S	read + data from CPU 4 (DSU)
5	**CPU 0 writes X**	I	M	BusUpgr
6	**CPU 4 writes X**	M	I	**RFO + data (CPU 4 was I)**

Steps 5 ⇄ 6 repeating = ping-pong (next slide). Step 6 is RFO, not BusUpgr: CPU 4's copy is Invalid, so it needs the data and the ownership.

Pattern: reads can be shared (steps 2, 4), but every write by the other core forces an invalidation (3, 5) or a full fetch with ownership (6).

Ping-Pong: When Coherence Gets Expensive

CPU 4 and CPU 0 write the same line in turn



Each write: invalidate the other copy + move the data back. Far slower than an L1 hit on one core.

Link to scheduling

Migration: the renderer moves big CPU 4 → LITTLE CPU 0. Its data stays in CPU 4's caches → misses and coherence traffic on CPU 0; moving back repeats the cost.

Placement and misfit move tasks between LITTLE and big — moving too often adds transfer cost.

Load balancing avoids migrating cache-hot tasks for exactly this reason.

What it costs

- more cache misses
- more cache-line transfers
- more DSU / interconnect traffic
- longer memory-access latency

No migration needed: two threads on two cores that keep writing the same line ping-pong too.

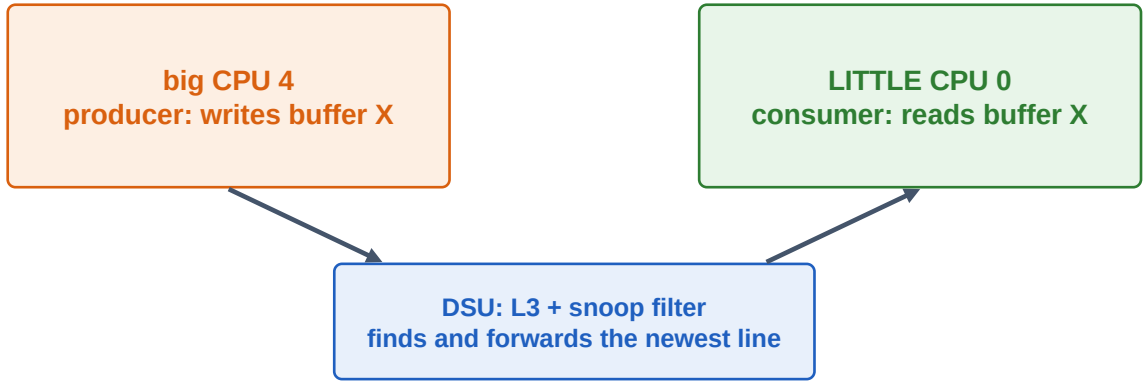
Remember: reads share, writes own (M); a write to a Shared line → BusUpgr; repeated writes from different cores → ping-pong.

Coherence is automatic, **not free**: frequent big ↔ LITTLE sharing or migration turns cache hits into cross-core transfers.

Coherence Meets big.LITTLE Placement

Where the scheduler places two communicating threads decides not only speed and power, but also **how far each shared cache line must travel**.

Producer on big, consumer on LITTLE (e.g. video data → follow-up work)



X travels CPU 4 → DSU → CPU 0: no DRAM trip, but still a transfer and coherence work.

Step		CPU 4	CPU 0	
1	**producer writes X**	M	I	CPU 4 holds the only, newest copy
2	**consumer reads X**	M	I	CPU 0 misses: no X yet
3	**DSU handles it**	M	I	CPU 4 is asked to supply the newest X
4	**consumer gets X**	S	S	both share it (basic MESI)

Producer / consumer placement	The line travels via	Cost	Trade-off
same CPU (take turns)	that core's L1 — no transfer	**lowest**	but no parallelism
two CPUs, one DynamIQ cluster	DSU L3 / snoop filter	**low – moderate**	runs in parallel; dearer than an L1 hit
big / LITTLE in different clusters	interconnect between clusters	**highest**	more latency and energy (classic big.LITTLE)

Qualitative comparison; real costs depend on the SoC.

PART 2: CPU Performance and Scheduling

- **Week 5**

- Cache Coherence: MESI on big.LITTLE
  - How do cores keep their private caches consistent?
- **Scheduling Fundamentals: MLFQ and Proportional Share**
  - Metrics → MLFQ (learn priority) → lottery & stride (share) → the bridge to Linux
- Scheduling Classes and Their Algorithms
  - 3-1 Deadline (EDF, CBS) → 3-2 Real-time (FIFO / RR) → 3-3 Fair (CFS, EEVDF)
- Scheduling Classes on webOS
  - Video (DEADLINE) → audio (FIFO, PI) → UI (EEVDF) → choosing a policy

Scheduling Basics: Two Metrics, No Single Winner

Turnaround time
`T = T_completion − T_arrival` → batch: finish soon

Response time
`T = T_firstrun − T_arrival` → interactive: start soon

Policy	Needs job length?	Turnaround	Response
FIFO	No	**Poor (convoy effect)**	**Poor**
SJF / STCF	**Yes — an oracle**	**Optimal**	**Poor**
Round Robin	No	**Poor (stretches every job)**	**Good**
MLFQ	**No — learns it**	**Near SJF for short jobs**	**Good**

Convoy effect: short jobs queue behind a long one. STCF: shortest time-to-completion first (SJF with preemption).

Question 1: Can we get good turnaround AND response without knowing job lengths?

→ **MLFQ — learn priority from behavior**

In Linux today: Linux RT class: strict priority levels

Question 2: Can we promise each job a fixed share of the CPU?

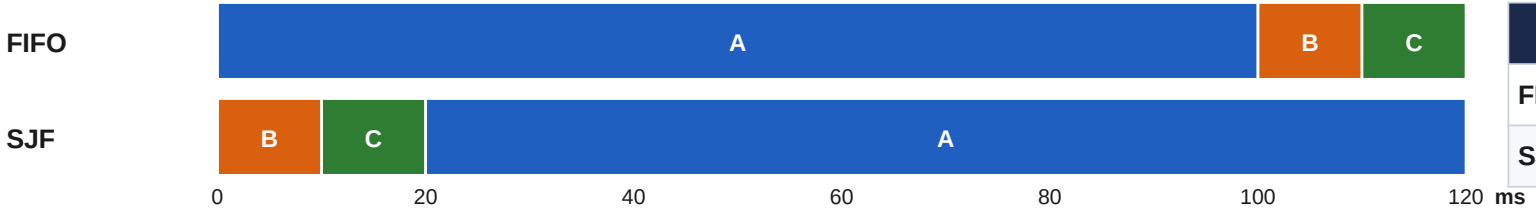
→ **Proportional share — lottery, stride**

In Linux today: Linux fair class: weights (CFS, EEVDF)

No policy wins both metrics. **MLFQ learns** priorities; **proportional share guarantees** shares — Linux uses both ideas.

Example: Computing Turnaround and Response

Example 1 — convoy: A = 100 ms, B = 10 ms, C = 10 ms, all arrive at t = 0

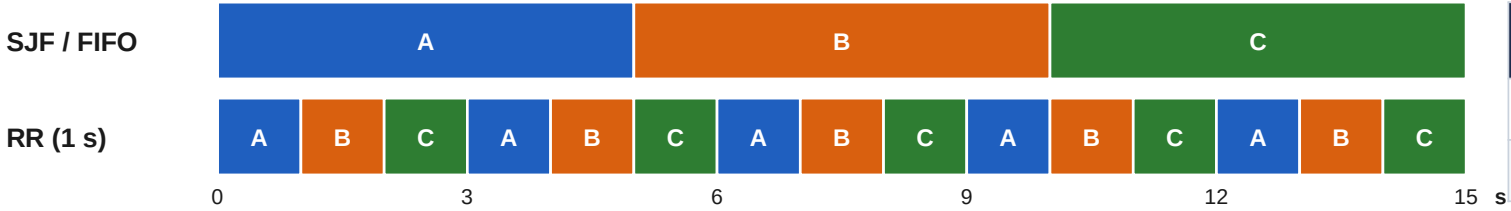


	finish times	avg turnaround	avg response
FIFO	100, 110, 120	110	70
SJF	10, 20, 120	50	10

Short jobs stuck behind a long one = convoy effect.

turnaround = finish − arrival; response = first run − arrival

Example 2 — equal jobs: A, B, C = 5 s each, all arrive at t = 0; RR time slice = 1 s



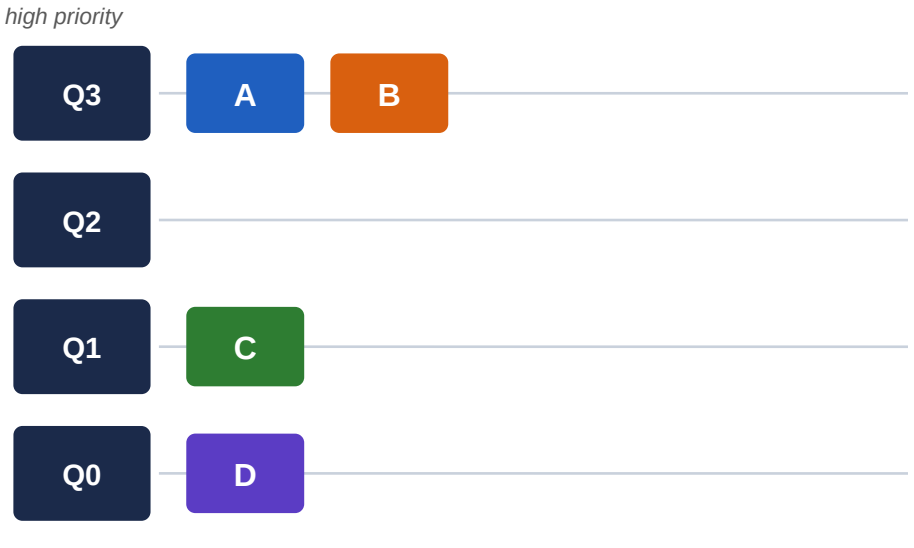
	first runs	finish	avg turnaround	avg response
SJF	0, 5, 10	5, 10, 15	10	5
RR	0, 1, 2	13, 14, 15	14	1

RR: great response, poor turnaround. A shorter slice improves response further but costs more context switches.

SJF wins turnaround but needs job lengths; RR wins response but stretches every job — **no single winner**.

MLFQ: Learn Priority from Behavior

Queues with priorities



A, B take turns at the top; C and D starve under Rules 1–2 alone.

Rule 1: If Priority(A) > Priority(B), A runs (B doesn't).
Rule 2: If Priority(A) = Priority(B), A and B run in round robin.

Priority comes from observed behavior

Interactive / I/O-bound

- Gives up the CPU often while waiting for keyboard or disk

→ keep priority **HIGH**

CPU-bound

- Uses the CPU for long stretches

→ **REDUCE** priority

The idea: learn from the past to predict the future — like branch predictors and caches. The scheduler never asks how long a job is; it watches, then guesses.
Risk: history can be wrong — jobs change phases, and some try to fool the scheduler.

Origin: F. J. Corbató (MIT), CTSS 1962 — one of the first time-sharing systems. 1990 Turing Award. MLFQ is over 60 years old and still in use.

MLFQ = **strict priority between queues + round robin inside**, with priority **learned from how each job behaves**.

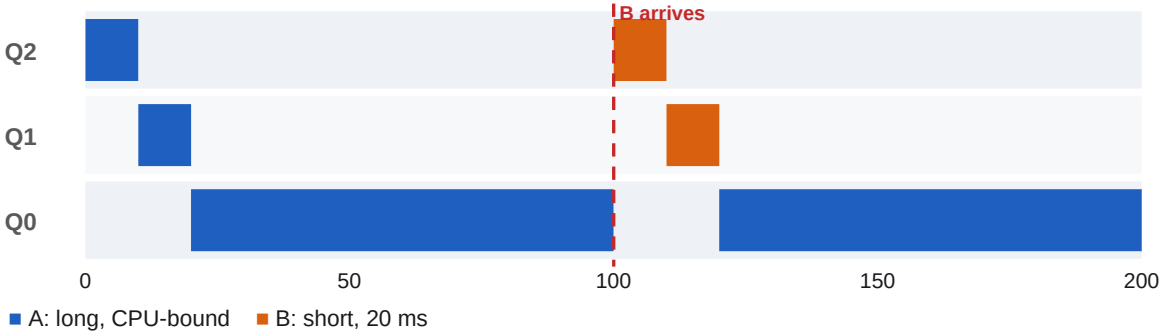
MLFQ Rules 3 and 4: Approximating SJF

Rule	When	Action
Rule 3	A job enters the system	Place it at the highest priority
Rule 4a	It uses up a whole time slice while running	Reduce its priority (move down one queue)
Rule 4b	It gives up the CPU before the slice ends	Stay at the same priority level

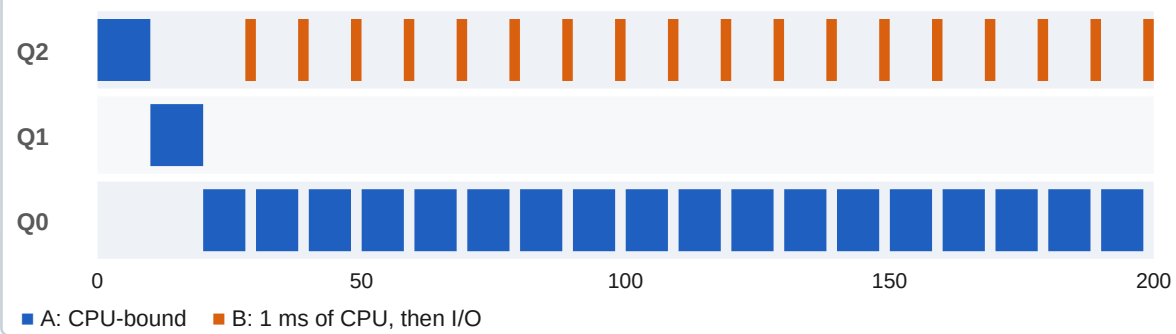
Why this approximates SJF

Assume every new job might be short → top priority. If it is short, it finishes at the top; if not, it sinks and reveals itself as a long batch job.

Along came a short job (3 queues, 10 ms slice)



An I/O-bound job stays on top (Rule 4b)



Short job

- B starts at once in Q2 and finishes within two slices
- A waits only 20 ms — turnaround of B close to SJF

Interactive job

- B always yields before its slice ends → never demoted
- Each time it wakes, it runs immediately → good response

Start high, **sink when you use a full slice** — short and interactive jobs stay on top without the OS knowing their length.

Basic MLFQ Breaks in Three Ways

1 Starvation

B, C (interactive) run 5 ms, wait 5 ms → never use a full slice → stay in Q2. Q2 is never empty, so **A in Q0 gets 0%**

Like a busy server's request threads starving a background job

Fix → Rule 5 (priority boost)

2 Gaming the scheduler

Run just under a slice, then block briefly. Rule 4b keeps the job on top: **B ≈ 90%, A ≈ 10%**

```
while (work_left) {
  compute(9 ms);   // just under 10
  usleep(1000);    // block -> yield
}
```

Must truly block (sleep, uncached read) — a cached write() returns at once.

Fix → new Rule 4 (allotment)

3 Behavior changes

An IDE builds (CPU-heavy → sinks to Q0), then turns interactive. Basic MLFQ never moves it back up: **each key press waits up to N × slice**

Typing feels laggy although the job is now interactive

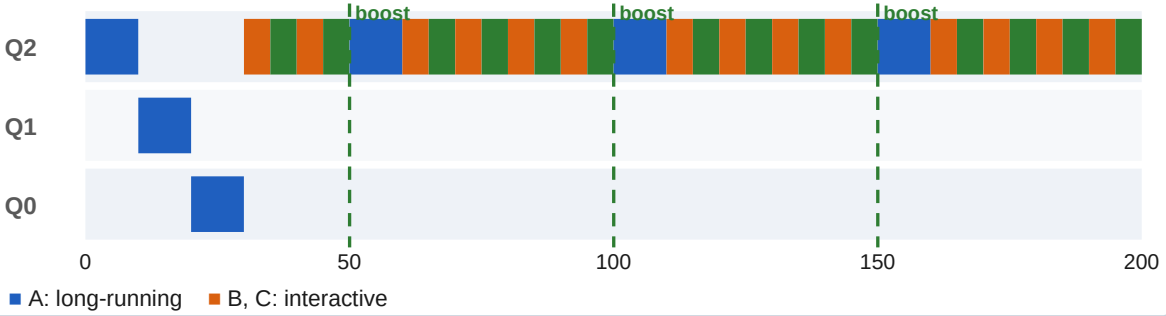
Fix → Rule 5 (priority boost)



Rules 1–4 can **starve** long jobs, can be **gamed**, and never notice a job that **changes phase**.

Fixing MLFQ: Priority Boost and Allotment Accounting

Rule 5: after some period S, move every job to the topmost queue.



New Rule 4: once a job uses up its allotment at a level — however many times it gave up the CPU — demote it.

Time (ms)	B does	Level	Used	Decision
50–59	run 9 ms	Q2	9/10	stay (old 4b would reset)
59–60	block 1 ms	Q2	9/10	kept across I/O
60–61	run 1 ms	Q2	10/10	**demote → Q1**
61–72	run, block, run	Q1	10/10	**demote → Q0**
72 –	RR with A	Q0	—	**B ≈ 47%, A ≈ 53% (was 90 / 10)**

What Rule 5 fixes — and the catch

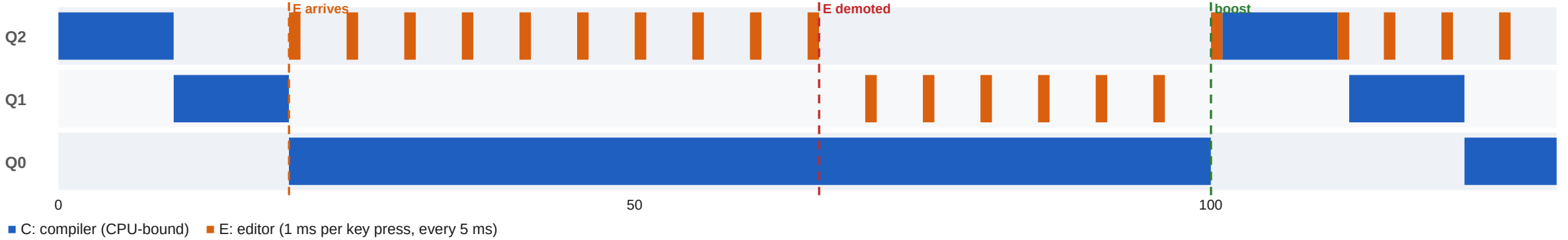
No starvation; phase changes are noticed after the next boost
S is a "voo-doo constant": too long → starvation; too short → interactive jobs lose share
Quick check: 10 ms slice, ≥ 5% for a long job → 10 / S ≥ 0.05 → **S ≤ 200 ms**

Old vs. new: old rules asked "did it use a full slice this time?"; the new rule tracks **total CPU used at this level**. The counter survives I/O and resets only on a level change or a boost — splitting work into small bursts no longer helps.

Final MLFQ rules
1 higher priority runs · **2** equal priority → RR · **3** new job → top · **4** allotment used up → move down · **5** every S → boost all to the top

Boost prevents starvation and catches phase changes; **allotment accounting** closes the gaming loophole.

Example: MLFQ Rule by Rule — A Compiler and an Editor

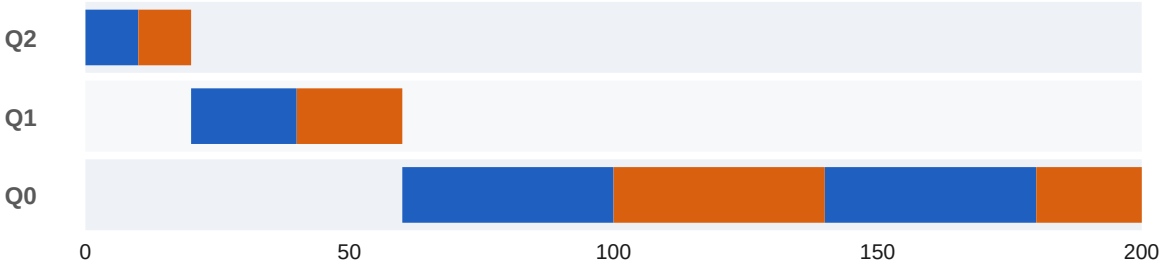


Time (ms)	What happens	C	E	Rule
0	C arrives → top queue	Q2	—	3
10, 20	C used its 10 ms allotment at Q2, then at Q1 → sinks	Q1 → Q0	—	4
20	E arrives → top queue	Q0	Q2	3
20 – 65	each key press: E (higher) preempts C at once	Q0	Q2	1
66	E's short runs add up to 10 ms at Q2 → demoted, although it always yielded	Q0	Q1	4
66 – 100	Q1 is still above Q0 → E still runs immediately	Q0	Q1	1
100	boost: everyone to the top — C cannot starve	Q2	Q2	5
100 – 111	same queue → round robin: E may wait for C's slice (the cost of a boost)	Q2	Q2	2

Together the five rules give the editor **fast response** and the compiler **steady progress** — with no job lengths given.

MLFQ in Real Systems: Tables, Formulas, Heuristics

Common practice: lower priority, longer slices



Two long jobs: **10 ms in Q2, 20 ms in Q1, 40 ms in Q0**

Solaris TS class: table-driven MLFQ

One row per priority level (60 levels): `ts_quantum` slice, `ts_tqexp` (Rule 4), `ts_slpret`, `ts_maxwait` (Rule 5)

- Slice 20 ms at the top → a few 100 ms at the bottom; boost ≈ 1 s

Admins tune it with `dispadmin -c TS -g` — no code change

FreeBSD 4.3: MLFQ by formula (details on the next slides)

`p_usrpri = 50 + ⌊estcpu / 4⌋ + 2 × nice`

lower value runs first; estcpu += 1 per tick while running

`each second: estcpu ← (2·load) / (2·load + 1) × estcpu + nice`

load 1 → × 2/3: a hog that turns to I/O recovers: 60 → 40 → 27 → 18

And Linux?

O(1) scheduler (2.6.0 – 2.6.22): guessed interactivity from sleep time and gave priority bonuses — MLFQ-style heuristics.

CFS (2007) dropped the guessing. Today's RT class keeps only Rules 1–2: strict priority, FIFO or RR within a level, no feedback.

Parameters to choose: number of queues · slice per queue · allotment per queue · boost period S

Lesson: every MLFQ needs tuned constants (Ousterhout's law: avoid voo-doo constants). Linux later replaced guessing with explicit requests — weights and slices (Part 3).

Same idea, different knobs: **tables (Solaris), formulas (FreeBSD), heuristics (old Linux)** — Linux RT keeps only the strict-priority part.

FreeBSD 4.3: MLFQ by Formula

Priority is recomputed by a **formula** — a continuous value, not a staircase of levels. It is recomputed every few clock ticks while a job runs, so priority drops in real time; the 32 run queues (value ÷ 4) are only buckets for fast selection.

1 Priority value — lower runs first

```
p_usrpri = PUSER
         + ⌊estcpu / 4⌋
         + 2 × nice
```

PUSER = 50, the best value a user process can get; 0–49 are for the kernel (range 0–127)
estcpu recent CPU usage — +1 per clock tick while running → value grows → runs later
nice user advice, −20 … +20

e.g., estcpu = 40, nice = 0 → 50 + 10 + 0 = 60

2 Decay (once per second)

```
estcpu ← factor × estcpu + nice
factor = (2·load) / (2·load + 1)
```

load load average (average number of runnable processes)
factor always < 1, so old usage fades every second
load ↑ factor nearer 1 → forgets more slowly (≈ 90% gone in 5 × load seconds)

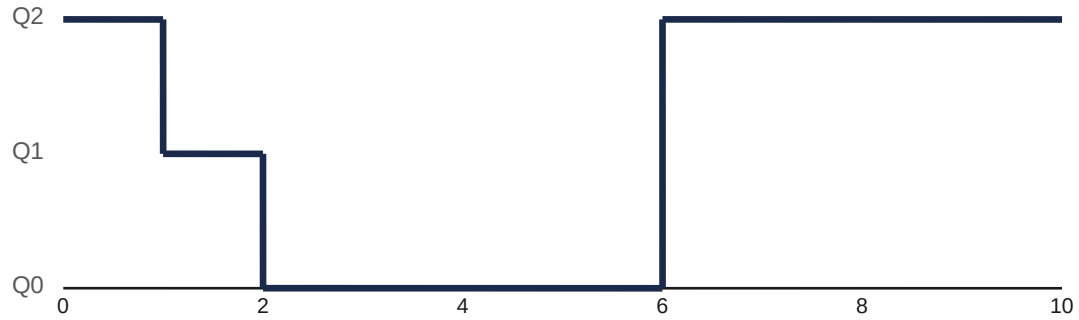
e.g., load = 1 → factor = 2/3: estcpu 60 → 40 → 27 → 18

Source: OSTEP §8.5; McKusick et al., 4.4BSD; Epema

Lower p_usrpri runs first (opposite of Solaris). With nice 0: estcpu 0 → 50 (best) · 40 → 60 · 120 → 80 (runs later).

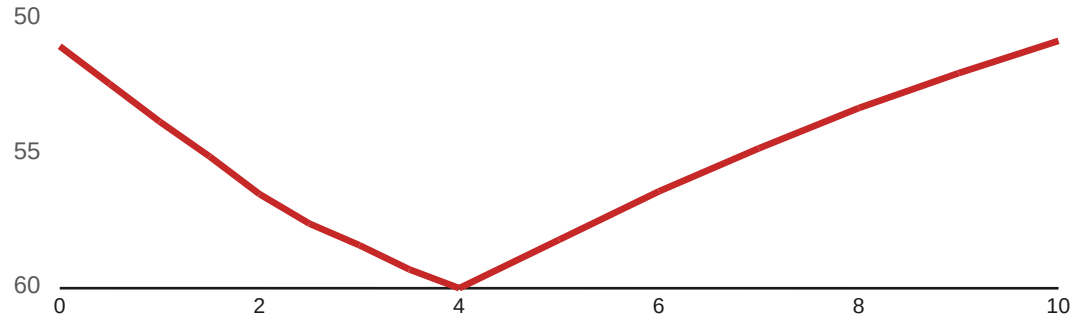
FreeBSD: The Formula Decides, Queues Only Hold

Textbook MLFQ — discrete levels (staircase)



jumps between queues by rules; the boost (t = 6 s) is one big jump

FreeBSD — continuous value (recomputed every few ticks)

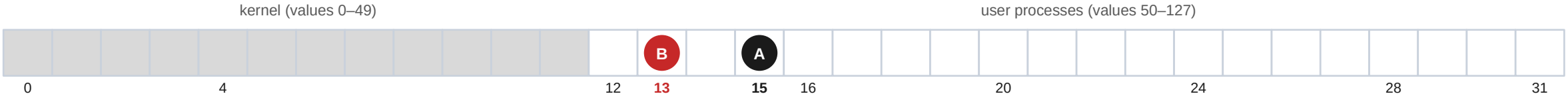


p_usrpri: 50 (top) = best. Running 0–4 s: rises tick by tick; waiting: decays each second (illustrative).

Kernel clocks:

every stat tick (≈ 8–10 ms) running job: estcpu += 1	every 4 ticks of CPU use p_usrpri recomputed	every 100 ms round-robin time slice	every 1 s decay all estcpu (schedcpu)

Internally: 32 run queues = buckets of 4 values (queue = ⌊value / 4⌋)



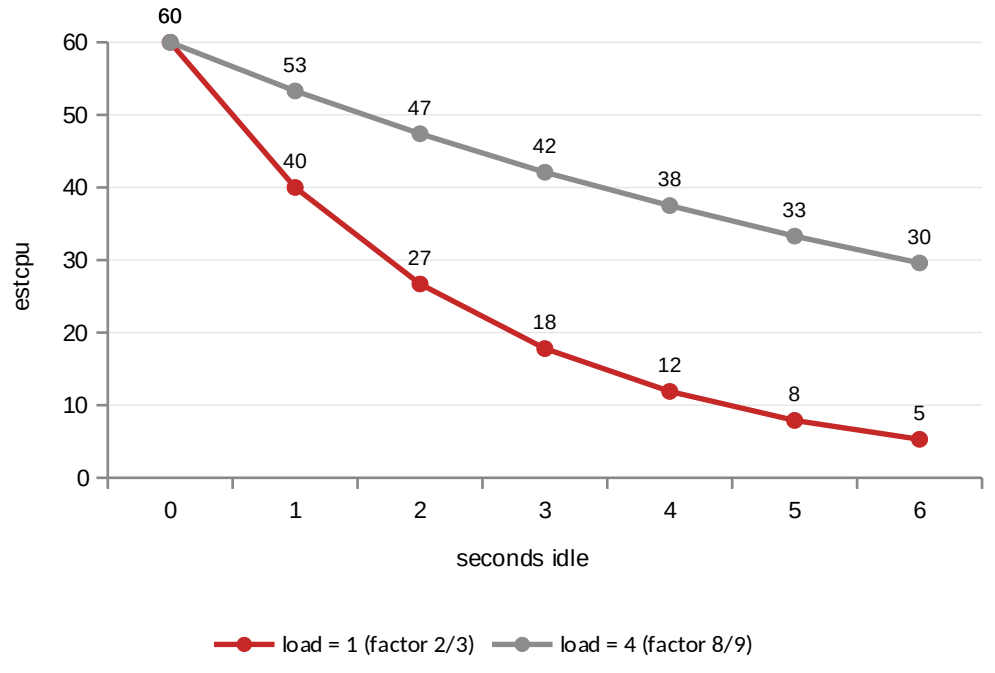
A: value 61 → queue 15 · B: value 54 → queue 13. **Scan from the best (left) end: the first non-empty queue (B) runs.**

Source: McKusick et al., 4.4BSD; curves illustrative

Policy lives in the formula; the queues are just a fast lookup structure.

FreeBSD: Decay in Action

A CPU hog (estcpu = 60) stops using the CPU and only waits for I/O



Worked example: load = 1 (factor 2/3), nice = 0

t (s)*	① estcpu × 2/3	② ⌊estcpu / 4⌋	③ p_usrpri
0	60 (start)	60 / 4 = 15 → 15	**50 + 15 = 65**
1	60 × 2/3 = 40.0	40.0 / 4 = 10.0 → 10	**50 + 10 = 60**
2	40.0 × 2/3 = 26.7	26.7 / 4 = 6.68 → 6	**50 + 6 = 56**
3	26.7 × 2/3 = 17.8	17.8 / 4 = 4.45 → 4	**50 + 4 = 54**
4	17.8 × 2/3 = 11.9	11.9 / 4 = 2.98 → 2	**50 + 2 = 52**
5	11.9 × 2/3 = 7.9	7.9 / 4 = 1.98 → 1	**50 + 1 = 51**

① decay each second (+ nice = 0) · ② round DOWN, not to nearest · ③ 50 + ② + 2 × 0

*** Why steps of 1 s?** Each row is one run of schedcpu(), which decays every estcpu once per second — a constant in the kernel source (like the 100 ms slice and 4-tick recompute), not admin-tunable like Solaris' table.

CPU hog = a CPU-bound process that keeps the CPU as long as it can (e.g., video encoding). Why does the grey line fade more slowly? → next slide.

Decay lets a former CPU hog **regain priority within seconds** — more slowly when the system is busy.

Why Decay More Slowly Under Load? — The Bucket Analogy

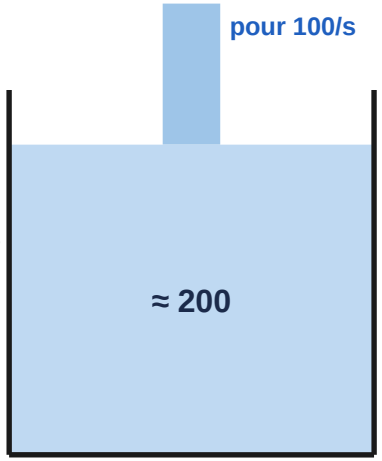
Water level = estcpu

Pouring = CPU time used (adds to estcpu)

Hole = decay (a share drains every second)

Scheduler full → CPU hog · low → interactive

1 Idle system (load = 1)

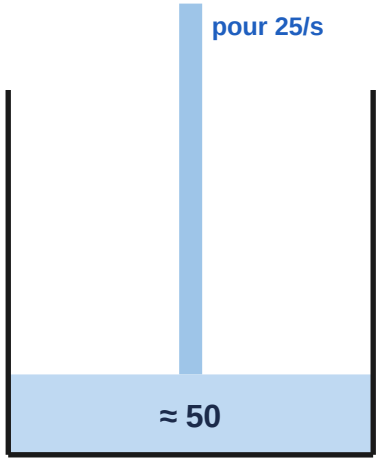


Hog runs all second

● drain 1/3 per s (keep 2/3)

Seen as CPU hog ✔

2 Busy (load = 4), same hole

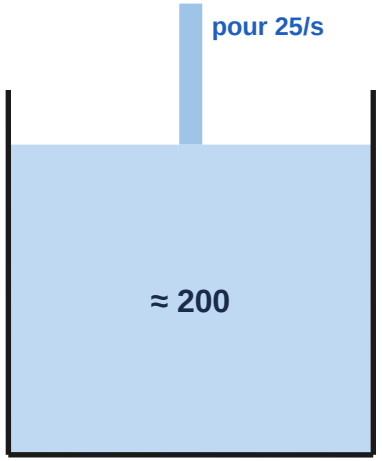


Hog gets only ~¼ of each second

● drain 1/3 per s (keep 2/3)

Mistaken for interactive ×

3 Busy (load = 4), smaller hole



FreeBSD: factor 8/9

● drain 1/9 per s (keep 8/9)

Seen as CPU hog ✔

"Remember longer" = earlier water stays in the bucket: after 5 s idle, keep 2/3 → 13% left; keep 8/9 → 55% left. **`Level ≈ pour × keep / (1 − keep).`** *Analogy and numbers constructed for illustration.*

Busy → a hog pours slowly, so FreeBSD **shrinks the hole (decays less)** to let the level fill up.

Proportional Share: Lottery Scheduling

MLFQ: optimizes turnaround / response, no promise on CPU amount. **Proportional share:** guarantees each job a set percentage.

Ticket = a unit of a resource a process should receive. Share = its tickets ÷ all tickets



Each slice: ① draw a random number 0 … 99 ② find who holds it ③ run that job for one slice

Textbook run: 20 draws

63	85	70	39	76	17	29	41	36	39	10	99	68	83	63	62	43	0	49	12
A	B	A	A	B	A	A	A	A	A	A	B	A	B	A	A	A	A	A	A

A ran 16 (80%), B ran 4 (20%) — target 75 / 25: close, not exact

Longer runs (simulated)

Slices	B's share	Error vs 25%
20	20.0%	**−5.0 pt**
100	25.0%	+0.0 pt
1,000	24.1%	−0.9 pt
10,000	24.7%	**−0.3 pt**

The longer jobs compete, the closer they get to 75 / 25 (law of large numbers).

No bad corner cases: no input always defeats a random choice

Little state: only a ticket count per job

Fast: one random number per decision

The price of randomness

Fairness holds only **on average**: over a short run a job can get much more or much less than its share. The ticket idea is general — the same mechanism works for memory or I/O bandwidth.

Source: OSTEP Ch. 9; Waldspurger & Weihl, OSDI '94.

Lottery: **tickets set the share**, a random draw picks each slice — proportional **on average**, not slice by slice.

Lottery: Ticket Mechanisms, Fairness and Limits

Currency

Each user hands out tickets in its own unit; the system converts to global tickets.

```
A: 100 global = 1000 local → A1 500, A2 500 → 50 + 50
B: 100 global = 10 local → B1 10 → 100
```

B1 gets 50% with only 10 local tickets.

Transfer

A client waiting for a local server lends its tickets to the server, which returns them with the reply.

```
Client 100 → 0, Server 10 → 110
```

Work done on the client's behalf runs faster.

Inflation

A process temporarily raises (or lowers) its own ticket count.

```
P1 50% → 75% after inflating
```

Safe only among trusting jobs; a greedy process could take the machine.

Unfairness metric `U = finish(first) / finish(second)`

Two jobs, equal tickets, 10 slices of work each. Perfectly fair → they finish together → U ≈ 1.



Lottery study (OSTEP Fig. 9.2): job length 1 slice → U = 0.5; ≈ 10 slices → U ≈ 0.8; only long jobs approach 1.

Short jobs can be treated very unfairly.

Limitation	Why	Fixed by
Short-term unfairness	random draws can favor one job for a while	**stride (next)**
Low predictability	outcome is probabilistic	—
Not real-time friendly	no bound on how long a job waits	—
O(n) per decision	walk the list until the ticket sum passes the winner	**tree: O(log n)**
Who assigns tickets?	the mechanism does not say	**CFS: the user, via nice**

Tickets are a flexible currency, but **randomness is unfair for short runs** — the motivation for a deterministic version.

Stride Scheduling: Proportional Share Without Luck

Job	Tickets	Stride = 10,000 / tickets
A	100	100
B	50	200
C	250	40

```
current = remove_min(queue);  // lowest pass
schedule(current);            // one quantum
current->pass += current->stride;
insert(queue, current);
```

Big stride = big step forward: few tickets → jump ahead in pass → wait longer.

pass A (+100)	pass B (+200)	pass C (+40)	Runs
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	all equal

After one cycle

A 2 · B 1 · C 5 runs

= 100 : 50 : 250 tickets

Exact every cycle — no luck involved. Lottery gets there only on average.

Weak spot: per-job state — what pass should a new job get?

A, B, C are at pass ≈ 10,000 when D (stride 100) arrives with pass 0 → **D runs 100 quanta in a row** until it catches up.



Fix: start a new job at the current minimum pass. **CFS does the same with min_vruntime.**

Lottery has no such problem: a new job just adds its tickets and joins the next draw.

Stride = lottery minus randomness: **run the lowest pass, add the stride** — exact shares; newcomers start at the minimum.

Example: Lottery vs. Stride on the Same Three Jobs

UI: 200 tickets → 50%	service: 100 → 25%	indexer: 100 → 25%

Lottery — draw 0 … 399: UI 0–199 · svc 200–299 · idx 300–399

57	312	145	388	21	333	190	76
UI	idx	UI	idx	UI	idx	UI	UI

After 8 draws: UI 5 · svc 0 · idx 3 (target 4 · 2 · 2)

Right on average, but in a short window the service may not run at all.

Draws are one illustrative random sequence.

Stride — L = 400: stride UI 2 · svc 4 · idx 4 (ties → UI, svc, idx)

#	pass UI	pass svc	pass idx	runs
1	0	0	0	UI
2	2	0	0	svc
3	2	4	0	idx
4	2	4	4	UI
5	4	4	4	UI
6	6	4	4	svc
7	6	8	4	idx
8	6	8	8	UI

After 8 runs: UI 4 · svc 2 · idx 2 — exactly 50 / 25 / 25

In Linux: tickets → weights. A 2 : 1 split ≈ nice 0 (1024) vs nice 3 (526) = 1.95 : 1, or cgroup cpu.weight 200 / 100 / 100.
CFS / EEVDF are deterministic like stride, so the short-term luck problem of lottery does not appear.

Same tickets, same target: **lottery is right on average**, **stride is right every cycle** — Linux chose the stride way.

From Lottery and Stride to Linux CFS

1994 Lottery
tickets + random draw

→ **1995 Stride**
same tickets, no randomness

→ **2007 Linux CFS**
stride for a real kernel

→ **2023 EEVDF**
keeps vruntime, adds deadlines

	Lottery	Stride	Linux CFS
Share input	tickets	tickets	**weight (from nice)**
A job's share	tickets ÷ Σ tickets	tickets ÷ Σ tickets	weight ÷ Σ weight
Per-job state	none	pass	**vruntime**
Picks next	random ticket winner	lowest pass	**lowest vruntime**
After running	nothing	pass += L / tickets	**vruntime += Δt × 1024 / weight**
New / waking job	just add tickets	start at min pass	**start at min_vruntime**
Data structure	list: O(n)	heap: O(log n)	red-black tree: O(log n)
Fairness	on average	exact each cycle	exact over time

`pass ≈ vruntime`

`stride ≈ 1024 / weight`

`min pass ≈ min_vruntime`

What CFS adds: it charges the time actually run (not a fixed quantum) and computes slices from the number of runnable tasks and their weights — details in Part 3-3.

CFS is stride scheduling for a real kernel: **pass → vruntime, tickets → weights, min pass → min_vruntime**.

From Theory to Linux: Where Each Idea Lives

Idea from theory	In Linux today	Where in this lecture
MLFQ Rules 1–2: strict priority, RR within a level	RT class: SCHED_FIFO / SCHED_RR (priority 1–99)	3-2
MLFQ feedback: guess interactivity from behavior	dropped after the O(1) scheduler; latency is now requested explicitly — EEVDF slice	3-3
MLFQ Rule 5: boost to stop starvation	RT throttling and the fair server keep fair tasks alive	3-2
Proportional share: tickets	fair class weights: nice → weight, cpu.weight	3-3
Stride: pass, start at min pass	vruntime, min_vruntime; EEVDF lag placement	3-3
Deadline-driven (beyond OSTEP Ch. 7–9)	SCHED_DEADLINE: EDF + CBS	3-1

Linux = strict priority on top, proportional share underneath



Next, Part 3: how each Linux class turns these ideas into an algorithm — **3-1 deadline, 3-2 real-time, 3-3 fair**.

PART 2: CPU Performance and Scheduling

- **Week 5**

- Cache Coherence: MESI on big.LITTLE
  - How do cores keep their private caches consistent?
- Scheduling Fundamentals: MLFQ and Proportional Share
  - Metrics → MLFQ (learn priority) → lottery & stride (share) → the bridge to Linux
- **Scheduling Classes and Their Algorithms**
  - 3-1 Deadline (EDF, CBS) → 3-2 Real-time (FIFO / RR) → 3-3 Fair (CFS, EEVDF)
- Scheduling Classes on webOS
  - Video (DEADLINE) → audio (FIFO, PI) → UI (EEVDF) → choosing a policy

Scheduling Classes and Their Algorithms

Class (high → low)	Policy	How the next task is picked
stop	— (kernel only)	No algorithm: one stopper thread per CPU (migration, hotplug) — next slide
deadline	`SCHED_DEADLINE`	EDF: earliest absolute deadline first; CBS: runtime budget per period
rt	`SCHED_FIFO`	Highest priority (1–99) first; runs until it blocks or a higher priority arrives
	`SCHED_RR`	Same as FIFO, but equal priorities take turns every time slice (100 ms)
fair	`SCHED_NORMAL`	**CFS (≤ 6.5): smallest vruntime · EEVDF (6.6+): eligible, earliest virtual deadline**
	`SCHED_BATCH`	Same CFS / EEVDF, but no wake-up preemption (throughput work)
	`SCHED_IDLE`	Same CFS / EEVDF with weight 3 (below nice 19 = 15) — not the idle class
idle	— (per-CPU idle task)	Runs only when nothing else is runnable → enters cpuidle

In this part (in class order)

3-1 SCHED_DEADLINE: EDF + CBS → 3-2 RT: FIFO / RR, throttling, PI → 3-3 fair: CFS → EEVDF

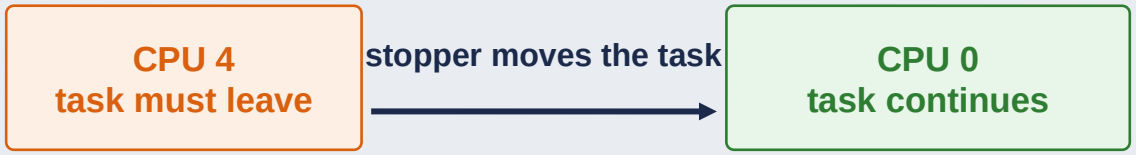


Linux first picks the **class**, then that class's own algorithm picks the task — **CFS / EEVDF live only in the fair class.**

The stop Class: CPU Stopper and CPU Hotplug

When is the stop class used?

Only the per-CPU **stopper threads** (**`migration/N`** in ps) — special kernel work that must own a CPU.



Uses: forced migration (active balance) · stop_machine · CPU hotplug

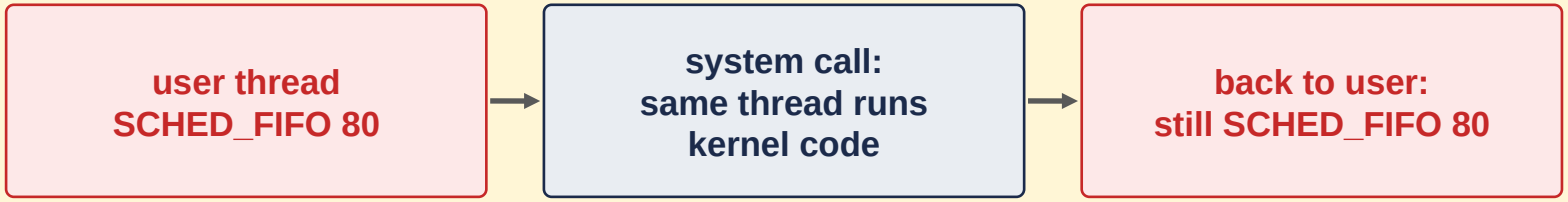
What is CPU hotplug?

Turning a core on or off while Linux keeps running. E.g., take big CPU 5 offline to save power:

```
# echo 0 > /sys/devices/system/cpu/cpu5/online
# echo 1 > /sys/devices/system/cpu/cpu5/online
```

Offline → its tasks move to other CPUs (the stopper thread helps). A logical on/off, not necessarily power-off.

Q: Does a system call turn my real-time thread into a kernel thread (or stop class)? No.



Kernel mode is a CPU privilege level, not a scheduling class.

Same task_struct, class and priority — not a kernel thread.

Kernel threads (kworker) are separate tasks, mostly fair class.

stop ≠ all kernel-mode code.

stop = the per-CPU stopper threads only (migration, hotplug); a system call never changes a thread's class.

SCHED_DEADLINE: Earliest Deadline First (EDF)

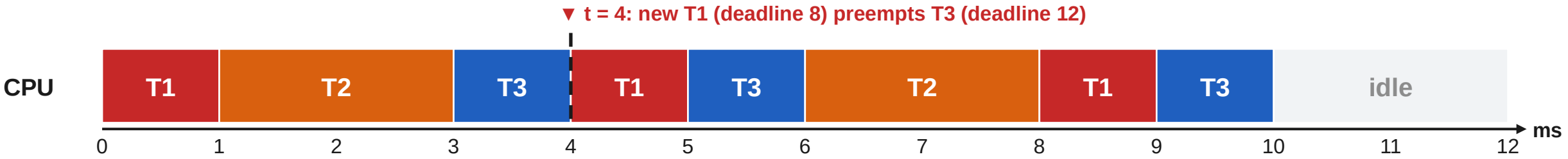
SCHED_DEADLINE = EDF: who runs first + **CBS (Constant Bandwidth Server)**: how much CPU each task may use

Task	Runtime C_i	Period T_i (= deadline)	Share C_i/T_i
T1	1 ms	4 ms	25%
T2	2 ms	6 ms	33.3%
T3	3 ms	12 ms	25%
Total			**83.3%**

Rule: run the ready job with the **earliest absolute deadline**

t = 0: deadlines T1 4 · T2 6 · T3 12 → **T1** → **T2** → **T3**

A new arrival with an earlier deadline **preempts** the running job.



T3 runs in pieces (3–4, 5–6, 9–10) = 3 ms, done before 12. Ties: the shorter period goes first.

Guarantee: $\sum C_i / T_i \le 1$ → EDF meets every deadline on one CPU (here 83.3% ✓). Assumes independent, preemptible periodic tasks, accurate runtimes, no extra blocking or overhead.

EDF orders deadline tasks by **when they must finish**, not by a priority number.

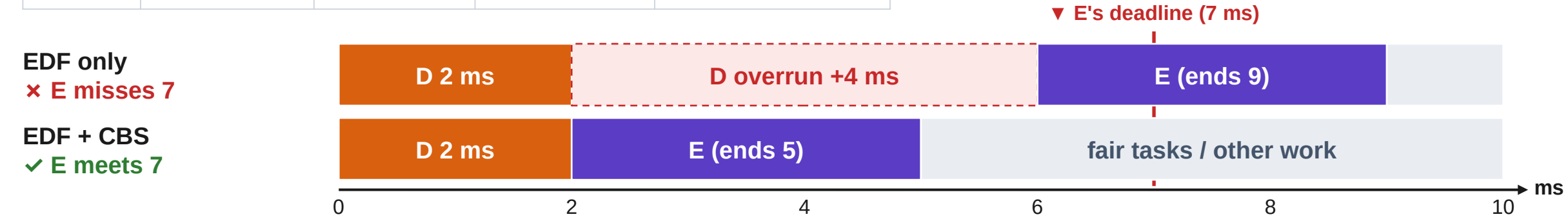
SCHED_DEADLINE: CBS Keeps Each Task in Its Budget

EDF picks who runs first, but does **not** stop overruns → **CBS (Constant Bandwidth Server)** enforces a CPU budget

Task	Runtime C_i	Period T_i	Deadline D_i	Bandwidth C_i/T_i
D	2 ms	10 ms	5 ms	20%
E	3 ms	10 ms	7 ms	30%
Total				**50%**

Problem: D reserved **2 ms**, but this time its job needs **6 ms**.

D has the earlier deadline (5 < 7), so EDF keeps running D.



CBS: D uses up its 2 ms budget → throttled until the budget is refilled next period. Only D is late.

```
# chrt -d --sched-runtime  2000000 \
          --sched-deadline 5000000 \
          --sched-period  10000000 0 ./task_D
```

Units: ns → runtime 2 ms, deadline 5 ms, period 10 ms (= task D)

Rule: **`Runtime ≤ Deadline ≤ Period`**

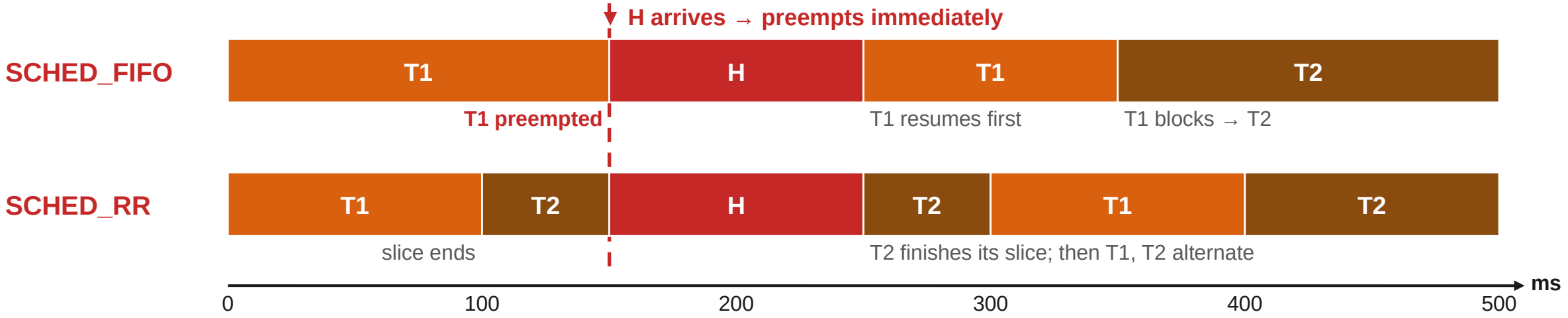
Admission control: total bandwidth checked; too much → rejected

SCHED_DEADLINE = EDF + CBS: EDF sets the order, CBS caps the CPU use and protects others.

Real-Time Policies: SCHED_FIFO and SCHED_RR

T1, T2: same RT priority 50. **H: priority 80** wakes at 150 ms.

RR time slice: 100 ms (sched_rr_timeslice_ms)



✔ Use RT for short, repeating, bounded work

Audio output — deliver audio data on time
Input handling — react quickly to input events
Watchdog — check system health on schedule

× Never for long or unbounded work

Big batch jobs, endless loops: a runnable RT task keeps **fair tasks (shell, UI)** off its CPU.

RT = strict priority, not "faster". A higher priority always preempts; keep RT work short and bounded.

Using RT: How to Set It and Who May Set It

1 How to set: per thread

In C (one thread):

```
struct sched_param p = {
    .sched_priority = 50 };   // RT priority (1–99)
pthread_setschedparam(tid,     // which thread
    SCHED_FIFO, &p);           // policy + priority
```

In the shell (a program):

```
$ chrt -f 50 ./audio_out
# run audio_out as SCHED_FIFO, priority 50
```

Policy and priority belong to **each thread**, not to the whole process.

2 Who may set it? Not only root

CAP_SYS_NICE

A Linux capability: the right to raise priority and set RT — one piece of root's power.

```
$ sudo setcap cap_sys_nice+ep ./audio_out
# audio_out may now set RT without root
```

RLIMIT_RTPRIO

A per-user limit: the highest RT priority a normal user may set (default 0 = none).

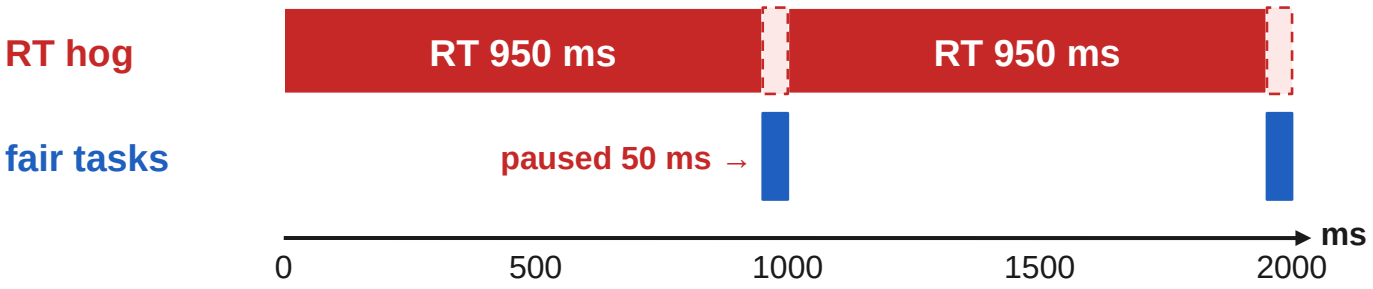
```
# /etc/security/limits.conf
@audio   -   rtprio   80    # audio group: up to 80
$ ulimit -r                  # check → 80
```



RT is set per thread and needs **CAP_SYS_NICE or RLIMIT_RTPRIO** — so a safety net is needed (next slide).

RT Throttling: A Safety Net for Runaway RT Tasks

CPU 2: `while (1) { }` in a SCHED_FIFO 80 task



Without throttling, the shell and UI on CPU 2 would never run.

How it works (per CPU)

1. Count RT run time in each 1 s period
2. Over 950 ms → the whole RT runqueue pauses, even prio 99
3. Fair tasks share the rest; none waiting → CPU idles
4. Refill every 1 s → RT resumes, still FIFO

Knobs (defaults)

```
sched_rt_period_us  = 1000000
sched_rt_runtime_us =  950000
```

Pauses RT; does **not** turn FIFO into NORMAL.
50 ms is for fair tasks as a group, not one task.
-1 disables it: never in production.

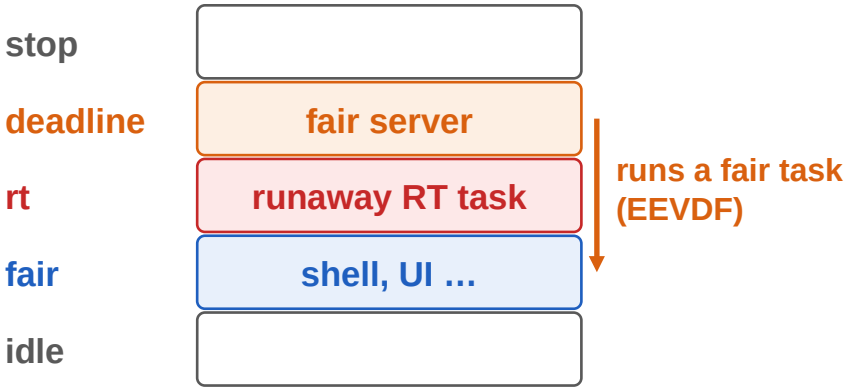
When it fires: find and demote the culprit

```
$ dmesg | grep -i throttl
[ 812.33] sched: RT throttling activated
$ ps -eLo tid,cls,rtprio,pcpu,comm --sort=-pcpu
  TID CLS RTPRIO %CPU COMMAND
  812  FF     50 95.1 audio_out
# chrt -o -p 0 812   # demote to normal
```

Throttling is a per-CPU **safety net**, not a design tool — if a real RT task hits it, fix the task.

Linux 6.12+: The Fair Server

Where it lives



Not a thread: a 50 ms / 1 s reservation

A RT hog, shell waiting → same result



B RT busy, no fair task waiting → classic wastes 50 ms



blue = fair tasks (50 ms)

50 ms is a budget, not a fixed slot

Fair tasks sleep after 10 ms → the server stops, RT resumes at once

Default 50 ms / 1 s = the classic 5%

Tunable per CPU (debugfs fair_server/cpuN)

During its 50 ms, who can preempt?

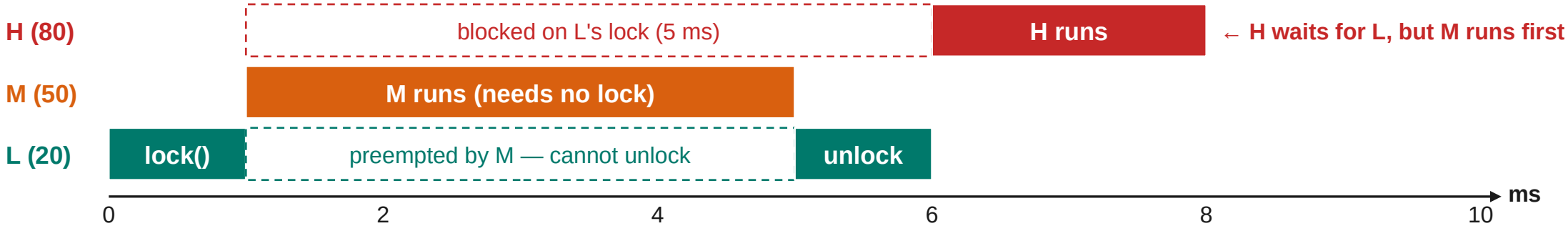
Who	Preempt?	Why
RT task (even 99)	**No**	deadline class is above rt
deadline task	**Yes**	EDF: earlier deadline first
another fair task	**Yes**	EEVDF shares the budget
stop class	**Yes**	always the top class

The fair server is a **reservation, not a pause**: fair tasks get up to 50 ms per second only when they need it.

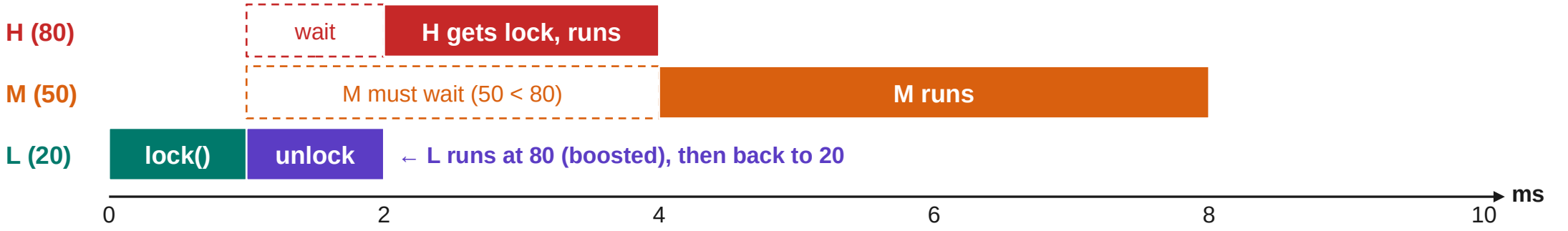
Priority Inversion and Priority Inheritance

Three RT tasks on one CPU: **H priority 80** urgent — **M priority 50** CPU work, no lock — **L priority 20** holds the lock

× Without priority inheritance: H waits 5 ms — 4 ms of it for M (inversion)



✓ With priority inheritance: L is boosted 20 → 80, H waits only 1 ms



The lock holder **borrows the waiter's priority** until it unlocks — so M can no longer delay H.

Using Priority Inheritance: a PI Mutex

Create the mutex once, with the PI protocol:

```
pthread_mutex_t lock;        // the shared lock
pthread_mutexattr_t a;       // its settings
pthread_mutexattr_init(&a);
pthread_mutexattr_setprotocol(&a,
        PTHREAD_PRIO_INHERIT);  // turn PI on
                  // default: PTHREAD_PRIO_NONE
pthread_mutex_init(&lock, &a);  // a PI mutex
pthread_mutexattr_destroy(&a);  // settings done
```

Then use it like any mutex:

```
pthread_mutex_lock(&lock);    // H blocks → L boosted
/* critical section: shared data, keep it short */
pthread_mutex_unlock(&lock);  // L drops back to 20
```

Priorities while H waits

Task	Original	During PI
H	80	80 (blocked)
M	50	50
L	20	**80 (boosted)**

Keep in mind

Temporary: L returns to 20 after unlock

Per mutex: only mutexes set to PRIO_INHERIT, not every lock

Short critical sections: a long hold still delays H

One line turns it on: **`PTHREAD_PRIO_INHERIT`** — the rest is an ordinary mutex.

How Does the Kernel Know Whom to Boost?

`mutex`

An ordinary mutual-exclusion lock: knows only "locked"

`futex`

Fast Userspace Mutex: lock word in user space; syscall only on contention

`rt_mutex`

Kernel lock that tracks the owner and waiters → PI

`pthread_mutex_lock()` → PI futex → kernel rt_mutex → find the owner → **boost the owner**

A PI lock word stores the owner's thread ID (e.g. owner TID = 812 = L) — that is how the kernel knows whom to boost.

1 L locks

owner = L, priority 20

2 H asks for it

waiter = H (80) → H blocks

3 Kernel checks rt_mutex

owner L, top waiter H at 80

4 Boost L

L: 20 → 80, so M cannot preempt L

5 L unlocks

critical section done

6 H runs, L restored

L back to 20 (unless another PI lock boosts it)

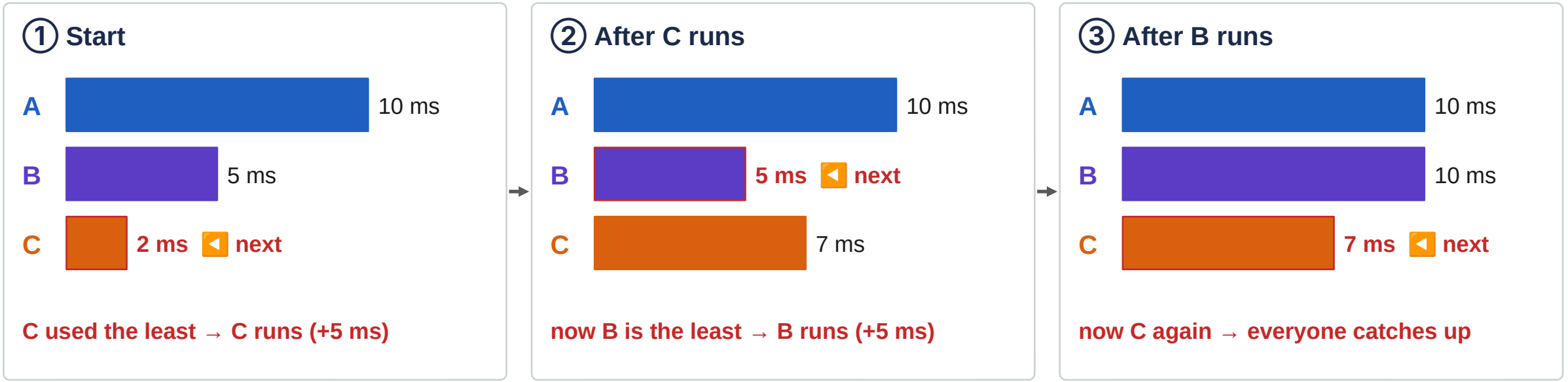
The kernel boosts **the owner recorded in the lock** — while L holds it for H, M cannot get in.

CFS: Run the Task That Has Used the Least CPU

2007 Linux 2.6.23: CFS becomes the default

2023 Linux 6.6: EEVDF replaces the pick rule

Example: three tasks, same priority, one CPU. Each bar = CPU time used so far. **Always pick the shortest bar.**



Same idea as stride (Part 2): pass → **vruntime**, tickets → **weight**, pick the smallest pass → **pick the smallest vruntime**. CFS compares a weighted clock, not raw time (next slides).

CFS = keep a clock per task, and **run the one that has used the least** — so everyone catches up.

CFS in Four Questions

1 How much CPU? → weight (from nice)

nice 0 = 1024. share = **`w / Σw`**
2048 : 1024 : 1024 → 50% : 25% : 25%

2 Who runs next? → smallest vruntime

`Δvruntime = Δt × 1024 / w`
10 ms used: nice −5 → 10 × 1024 / 3121 ≈ 3.3
nice 0 → 10 × 1024 / 1024 = 10, nice 5 → 10 × 1024 / 335 ≈ 30.6

3 How to find it fast? → red-black tree

Runnable tasks sorted by vruntime;
pick the leftmost: O(1); insert / remove: O(log N)

4 How long does it run? → slice from a period

`slice = P × w / Σw` (P = 24 ms)
equal 1024 × 3 → 24 / 3 = 8 ms each
2048 : 1024 : 1024 → A = 24 × 2048 / 4096 = 12, B = C = 6 ms

Sleep and wake-up: a sleeping task's vruntime does not grow, so a UI task that wakes up often runs soon — but CFS places it near **`min_vruntime`**, so it cannot save unlimited credit.



Weight decides **how much**, the smallest vruntime decides **who**, the period decides **how long**.

nice → weight: How Much CPU Do You Get?

nice	weight	vs nice 0	share vs one nice-0 task
−20	88761	≈ 87×	**98.9%**
−10	9548	≈ 9.3×	**90.3%**
−5	3121	≈ 3.0×	**75.3%**
−1	1277	≈ 1.2×	**55.5%**
0	1024	1×	**50.0%**
1	820	≈ 1/1.2	**44.5%**
5	335	≈ 1/3.1	**24.7%**
10	110	≈ 1/9.3	**9.7%**
19	15	≈ 1/68.3	**1.4%**

Geometric: 5 nice steps ≈ 3 : 1 anywhere — 1024 / 335 ≈ 110 / 36 ≈ 3.06. Extremes: 88761 / 15 ≈ **5,900 : 1**

weights: kernel/sched/core.c (sched_prio_to_weight)

CHECKPOINT

UI and indexer, both nice 0, share one core → 50% : 50%. Indexer moved to nice 10 → UI gets 1024 / (1024 + 110) ≈ **90%**, indexer ≈ 10%.

```
$ nice -n 10 ./indexer          # start with lower weight
$ renice -n 5 -p 1234           # change a running task
// C: setpriority(PRIO_PROCESS, tid, 10);
# echo 400 > /sys/fs/cgroup/ui/cpu.weight   # group: 100 = nice 0
```

Raising priority (negative nice) needs root / CAP_SYS_NICE.

Terms

nice — −20 … 19, the user-visible value; "nicer" (higher) = gives CPU to others
weight — the kernel's number behind nice; nice 0 = 1024
sched_prio_to_weight — the fixed kernel table that maps the 40 nice values to weights
cpu.weight — cgroup v2 weight for a whole group, 1 … 10000; 100 = nice 0
CAP_SYS_NICE — the privilege needed to raise priority (lower nice)

Why is each nice step ×1.25?

Goal: One nice step ≈ 10 points of CPU: nice 0 vs 1 → 1024 : 820 = 55.5% : 44.5%.
Why multiplicative? Only the difference matters: nice 0 vs 5 and nice 10 vs 15 both split ≈ 75 : 25.
Caution: nice sets how much CPU, not how soon (→ EEVDF slice).

Weight sets the share: `share = w / Σw`, and each nice step changes weight by about ×1.25.

One Algorithm, Three Policies: NORMAL, BATCH, IDLE

`SCHED_NORMAL` weight from nice · may preempt on wake-up

`SCHED_BATCH` weight from nice · never preempts on wake-up

`SCHED_IDLE` fixed weight 3 · never preempts, yields to NORMAL

policy = per-task setting

fair class: one run queue, one algorithm

CFS (≤ 6.5)
smallest vruntime

EEVDF (6.6+)
eligible + earliest deadline

algorithm = how the class picks — the same for all three policies

Policy (ps CLS)	Weight	Wake-up preemption	Typical use	Set with
SCHED_NORMAL (TS)	from nice: 15 … 88761	**yes — may preempt current**	apps, UI, services (default)	`(default) nice / renice`
SCHED_BATCH (B)	from nice	**no — waits for current's turn to end**	compile, indexing, backup	`chrt -b 0 ./make`
SCHED_IDLE (IDL)	**fixed 3 (nice ignored)**	**no — and any NORMAL wake-up preempts it**	leftover-time cleanup, prefetch	`chrt -i 0 ./cleanup`

Example: UI (NORMAL, 1024) vs cleanup (IDLE, 3) → UI 1024 / 1027 = **99.7%**; when UI sleeps, cleanup gets 100%. As nice 19 (weight 15) it would take 1.4%.

Terms

policy — per-task setting (sched_setscheduler / chrt) that selects a class and its flavor
algorithm — the class's pick rule: CFS until 6.5, EEVDF from 6.6 — shared by all three
wake-up preemption — a task that wakes up may take the CPU from the running one
SCHED_IDLE ≠ idle class — IDLE is a fair policy (weight 3); the idle class only runs the per-CPU idle task

Why?

BATCH? Throughput jobs gain nothing from preempting on wake-up; fewer switches = warmer caches.
IDLE? Run only in leftover time: nice 19 still takes 1.4% from the UI, IDLE ≈ 0.3%.
One algorithm? All three share one vruntime clock, so they stay comparable and no task can starve forever.

Policy sets **weight and wake-up behavior**; the **algorithm (CFS / EEVDF)** is the same for NORMAL, BATCH and IDLE.

CFS Core Idea: vruntime, a Weighted Clock

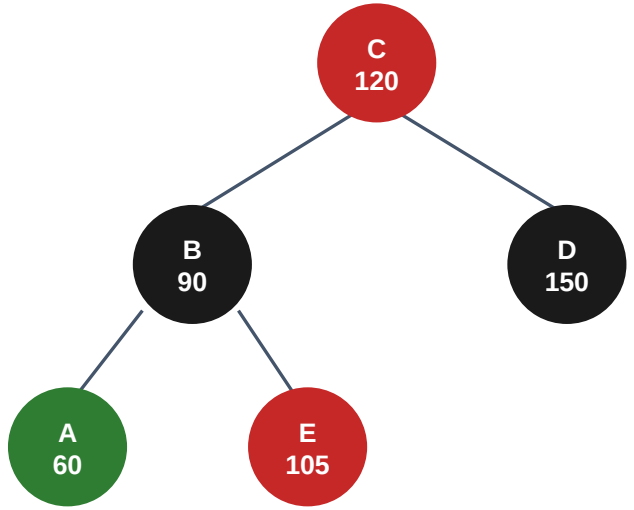
```
vruntime += Δexec × 1024 / weight
```

Same 3 ms of real CPU time →

task	weight	vruntime grows
nice −5	3121	**0.98 ms**
nice 0	1024	**3.0 ms**
nice 5	335	**9.2 ms**

- Heavy task: clock runs slow → looks "behind" → picked more often
- Light task: clock runs fast → soon looks "ahead"

Run queue: red-black tree by vruntime



← leftmost (cached) = next to run

pick O(1) · insert / remove O(log N)

When does CFS switch?

Tick: current used up its slice (next slide) → need_resched

Wake-up: woken task's vruntime is smaller by more than wakeup_granularity (1 ms × CPU factor) → preempt

Block / yield: current sleeps → pick leftmost now

Terms

vruntime — virtual runtime: CPU time used, scaled by 1024 / weight (ns)
red-black tree — a self-balancing sorted tree; here sorted by vruntime
leftmost — the node with the smallest vruntime = the next task to run
min_vruntime — follows the smallest vruntime in the queue and never decreases
need_resched — a flag: "switch tasks at the next safe point"

Why?

A weighted clock? One rule — run the smallest vruntime — then yields shares ∝ weight, with no priority levels.
A sorted tree? Many runnable tasks; insert / remove in O(log N) and always know the minimum.
Cache the leftmost? Picking the next task is the hottest path in the scheduler: make it O(1).

CFS = a weighted clock per task + **"run the smallest vruntime"**; weight decides how fast the clock runs.

CFS: How Long Does Each Pick Run?

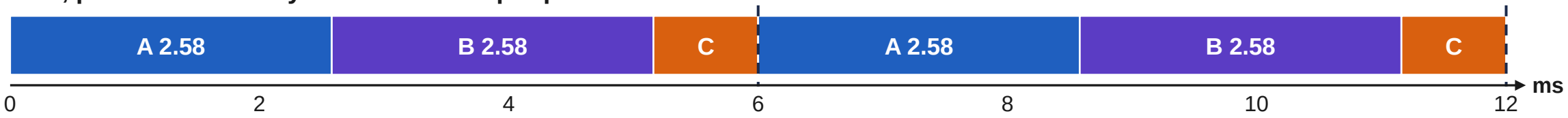
Example: 3 runnable tasks on one CPU — **A** nice 0 (w = 1024), **B** nice 0 (w = 1024), **C** nice 5 (w = 335)

1. **period = max(sched_latency, n × min_granularity)**
   `= max(6 ms, 3 × 0.75 ms) = 6 ms  (n = 3)`
2. **Σw = sum of weights of runnable tasks**
   `= 1024 + 1024 + 335 = 2383`
3. **$slice_i = period \times w_i / \Sigma w$**
   `= 6 ms × wᵢ / 2383  (table →)`

task	weight	slice = 6 × w / 2383	Δvruntime per pick
A	1024	2.58 ms	2.58 × 1024 / 1024 = 2.58
B	1024	2.58 ms	2.58 × 1024 / 1024 = 2.58
C	335	0.84 ms	0.84 × 1024 / 335 = 2.58

Same Δvruntime for all → heavier tasks are not picked more often; they run longer each time.

One CPU, period 6 ms: every task runs once per period



Many tasks? Stretch the period
Keep each slice ≥ 0.75 ms: 20 tasks → **20 × 0.75 = 15 ms**

How sched_latency and min_granularity are set
6 ms / 0.75 ms × (1 + $\log_2$ CPUs), rounded down, max 8 CPUs:
1 CPU ×1 · 2 ×2 · 4–7 ×3 · 8+ ×4. 6 CPUs → 18 ms / 2.25 ms

`slice = period × w / Σw`: every runnable task runs once per period, heavier ones for longer.

Why EEVDF? "How Much" Is Not "How Soon"

2007 Linux 2.6.23: CFS → **2023** Linux 6.6: EEVDF replaces the pick rule (paper: 1995)

Problem: CFS has only one knob — weight

A UI renderer: short bursts, needs a **fast response**
B batch job: long runs, needs **throughput**
Both nice 0 → CFS sees only 50% : 50%
Lower A's nice? A runs sooner — but **also takes more CPU**
So latency was patched with heuristics (sleeper bonus, wakeup_granularity).

EEVDF: add a second knob — slice

weight → how much CPU (unchanged)
slice → how soon it runs
Pick: eligible (lag ≥ 0), then the earliest virtual deadline
`VD = start + slice / weight`
Short slice → picked sooner, not more CPU.

	CFS (2.6.23 – 6.5)	EEVDF (6.6 +)
How much CPU	weight (nice, cpu.weight)	**same**
Who runs next	smallest vruntime	**eligible + earliest virtual deadline**
How soon	no knob — heuristics	**slice**
After waking	sleeper bonus (guess)	**lag is kept**



CFS asks **"who used the least?"** EEVDF also asks **"who must run soonest?"**

EEVDF in Two Steps: Who May Run, Then Who Runs First

1 **Step 1 Lag** — Fairness

Who is **allowed** to run? → lag ≥ 0

2 **Step 2 Virtual deadline** — Latency

Of those, who runs **first**? → earliest VD

Example (equal weight)

Runnable tasks	Step 1: lag = V − v ≥ 0 ? (V=15)	Step 2: earliest VD ?	Run
A (UI) v = 13 B (batch) v = 12 C v = 20	A +2 ✓ B +3 ✓ C −5 × waits	A VD = 13 + 2 = 15 B VD = 12 + 10 = 22 (slice: A 2 ms, B 10 ms)	**A (UI)** fair and responsive

CFS (one step): smallest vruntime → **B (12) runs**, the UI task A waits — CFS cannot see that A needs a fast response.

Where does V = 15 come from? Nobody sets it — the kernel computes it

V = weighted average vruntime of the runnable tasks = $\Sigma\ w_i \cdot v_i\ /\ \Sigma\ w_i$

Equal weights: $V = (13 + 12 + 20) / 3 = 15$ → $lag_i = V - v_i$: A +2, B +3, C −5

V moves forward as tasks run, and is updated when a task joins or leaves the queue.

Step 1 keeps it **fair** (who may run); Step 2 makes it **responsive** (who runs first).

PART 2: CPU Performance and Scheduling

- **Week 5**

- Cache Coherence: MESI on big.LITTLE
  - How do cores keep their private caches consistent?
- Scheduling Fundamentals: MLFQ and Proportional Share
  - Metrics → MLFQ (learn priority) → lottery & stride (share) → the bridge to Linux
- Scheduling Classes and Their Algorithms
  - 3-1 Deadline (EDF, CBS) → 3-2 Real-time (FIFO / RR) → 3-3 Fair (CFS, EEVDF)
- **Scheduling Classes on webOS**
  - Video (DEADLINE) → audio (FIFO, PI) → UI (EEVDF) → choosing a policy

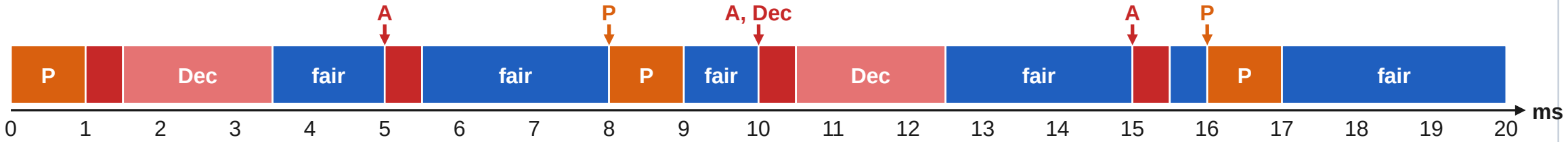
Example: Five Threads, One CPU — Who Runs When?

Thread	Policy	Needs
Frame pacer	SCHED_DEADLINE	1 ms every 8 ms
Audio	SCHED_FIFO 80	0.5 ms every 5 ms
Decoder	SCHED_FIFO 50	2 ms every 10 ms
UI	NORMAL, nice 0 (w 1024)	always runnable
Indexer	NORMAL, nice 10 (w 110)	always runnable

Who runs? Class first, then its algorithm

1 **DEADLINE** — earliest deadline (EDF)
2 **FIFO / RR** — higher priority (80 > 50)
3 **fair** — share by weight (EEVDF)
A higher class always preempts a lower one.

One CPU, 0 – 20 ms ■ P pacer ■ A audio ■ Dec decoder ■ fair = UI + indexer ▼ **new period → preempts**



In 20 ms	CPU time
Deadline (1 ms × 3)	3 ms
RT: audio 0.5 × 4 + decoder 2 × 2	6 ms
Fair: left over for UI + indexer	11 ms

UI 1024 / (1024 + 110) ≈ 90.3% of 11 ms ≈ **9.9 ms**
Indexer 110 / 1134 ≈ 9.7% ≈ **1.1 ms**
UI gets 90% of the leftover — only ≈ 49.5% of all 20 ms.

Class first, then the class's rule: **deadline > RT > fair**; fair tasks share what is left by weight.

Example: Sizing SCHED_DEADLINE for webOS Video

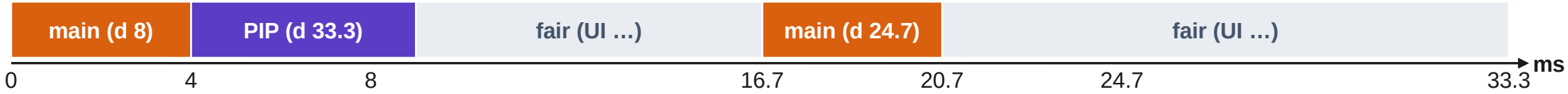
60 fps = 60 frames per second → time per frame = **`1000 ms / 60 ≈ 16.7 ms`** → **period = 16.7 ms**

Stream	runtime	deadline	period	bandwidth = runtime / period
Main video 60 fps	4 ms	8 ms	16.7 ms	4 / 16.7 ≈ 24%
PIP 30 fps	5 ms	33.3 ms	33.3 ms	5 / 33.3 ≈ 15%
Total				**24% + 15% = 39% ≤ 95% ✔**
+ 4K HDR stream	12 ms	16.7 ms	16.7 ms	**39% + 12 / 16.7 (72%) = 111% ×**

Where 95% comes from

Linux default RT / deadline bandwidth:
`950000 / 1000000 µs`
(sched_rt_runtime_us / sched_rt_period_us).
Over 95% → EBUSY. Here: all streams on one CPU.

EDF on one CPU: both streams wake at 0



runtime too small (needs 4 ms, set 2 ms): CBS stops it at 2 ms → the rest misses the frame → **frame drop**

runtime too big (needs 4 ms, set 12 ms): reserves 72% → the next DEADLINE task is **rejected by admission**

period = frame time, runtime = measured worst case — **not too small (drops), not too big (rejections)**.

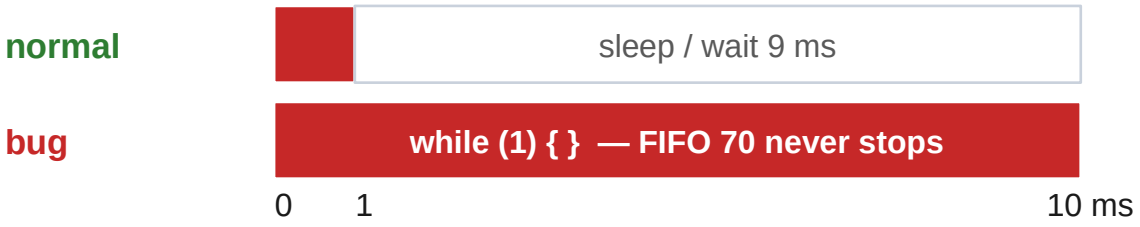
Example: webOS Audio — FIFO Priorities and Throttling

Thread	Work	Policy	Why
Audio output	0.5 ms every 5 ms	**FIFO 80**	device needs a buffer every 5 ms; late → glitch
Mixer	1 ms every 10 ms	**FIFO 70**	mixes streams; longer period → lower priority
Decoder	1 ms … 10+ ms	**fair (NORMAL)**	long and variable → must not be RT

RT use: `0.5 / 5 + 1 / 10 = 10% + 10% = 20%` **≪ 95% (default: RT may run 950 ms of every 1000 ms)**

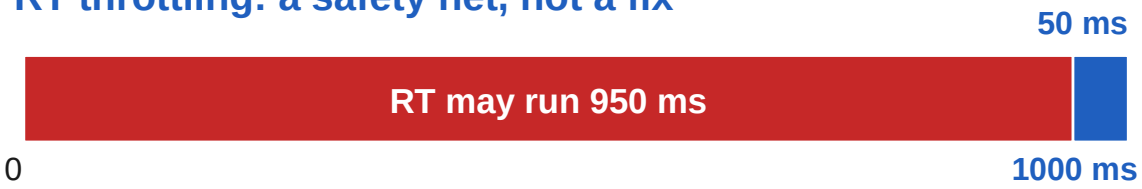
Priority is not only "shorter period → higher": real systems also weigh dependencies and deadlines.

What if the mixer never sleeps?



→ **UI and decoder (fair) on that CPU get almost no CPU.**

RT throttling: a safety net, not a fix



Blue 50 ms: fair tasks run. But audio output (FIFO 80) on that CPU is paused too → **glitch anyway.**

Audio RT: **short period → higher priority, do the work and sleep every period**; throttling only stops a total freeze.

Example: webOS Audio — Priority Inversion and PI

Mixer (FIFO 70) and decoder (fair) share one audio buffer, guarded by a mutex.

× Without PI: inversion

1. Decoder takes the mutex
2. Mixer (FIFO 70) asks for it → blocks
3. Another RT task runs → the fair decoder cannot run
4. Mixer keeps waiting on a lower-priority task

✓ With PI (PTHREAD_PRIO_INHERIT)

1. Mixer blocks on the decoder's mutex
2. Kernel boosts the decoder to FIFO 70
3. Decoder finishes and unlocks quickly
4. Boost dropped; mixer takes the lock and runs

	Mixer	Decoder while holding the lock	Result
No PI	FIFO 70	**fair — can be preempted**	**unlock may be delayed**
PI	FIFO 70	**FIFO 70, inherited for now**	**a chance to unlock quickly**

```
pthread_mutexattr_setprotocol(&a,
PTHREAD_PRIO_INHERIT);
```

PI does not shorten the work: keep the critical section short.



Shared lock between RT and fair → **use a PI mutex and keep the critical section short**.

Example: webOS UI on EEVDF — Weight vs. Slice

How much — weight from nice (only when they compete)

Thread	nice	weight	share
Compositor (LSM)	−5	3121	**73%**
App renderer	0	1024	24%
Indexer	10	110	2.6%

`share = w / Σw,  Σw = 3121 + 1024 + 110 = 4255`

How soon — a key press wakes both; both eligible

`VD = V + slice × 1024 / w`

Compositor: 1 ms × 1024 / 3121 → **V + 0.33** ← **runs first**

Renderer: 2.25 ms × 1024 / 1024 → V + 2.25

A short slice runs sooner, not longer.

Indexer: SCHED_BATCH or SCHED_IDLE — it never preempts the UI on wake-up and only fills leftover time.
Share ≠ latency: more weight = more CPU under contention; a shorter slice = picked sooner.

```
[Service]   # lsm.service
Nice=-5           # or cgroup cpu.weight
```

```
struct sched_attr a = { ... };
a.sched_runtime = 1000000; // 1 ms slice, 6.12+
```

On webOS UI: **weight = how much, slice = how soon, BATCH / IDLE = out of the way**.

Choosing a Policy: Fair, RT or Deadline?

Who runs first (class order) — first

SCHED_DEADLINE
real deadline + CPU budget

SCHED_FIFO / RR
static RT priority 1–99

SCHED_NORMAL
fair: weight + EEVDF

	Fair (EEVDF)	RT (FIFO / RR)	Deadline
Picks by	weight + virtual deadline	static RT priority	real (absolute) deadline
CPU split	by weight	highest priority first	runtime budget
Response	better, not guaranteed	good for high priority	timing is expressed
Starvation	**low**	**high (throttling helps)**	**low (budget + admission)**
Typical	UI, browser, services	audio, control loops	periodic media work

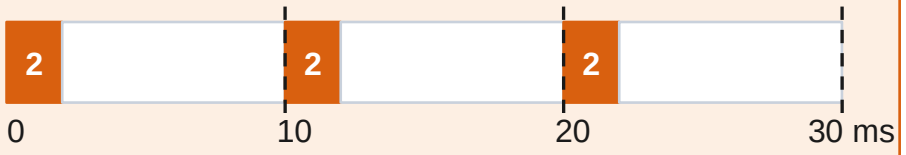
Share ≠ priority

UI at nice −20 (huge weight) still waits for any runnable FIFO task.

FIFO priority 1 beats NORMAL nice −20.

nice/weight only matter inside the fair class.

Deadline example: 2 · 10 · 10 ms



runtime · deadline · period → 2 ms budget every 10 ms (EDF + CBS).

Real deadline ≠ EEVDF's virtual deadline.

Linux decides in 3 steps

1 Class: DL → RT → fair

2 Task: EDF · FIFO/RR · EEVDF

3 Time: budget · priority · weight

Pick the class by the need: **fair share** · **strict priority** · **real deadline** — weight never beats a higher class.

Summary: How Linux Decides and Coordinates

1 Part 1 · Cache coherence

- Snooping / DSU keep private caches consistent
- MESI: reads share, a write owns the line (M)
- Shared writes move lines → placement matters

2 Part 2 · Scheduling theory

- MLFQ learns priority from behavior
- Lottery / stride split the CPU by tickets
- → weight and vruntime in CFS

3 Part 3 · Classes and algorithms

- Class first: deadline > RT > fair
- EDF + CBS · FIFO / RR + throttling, PI
- CFS → EEVDF: weight = how much, slice = how soon

4 Part 4 · On webOS

- Video: DEADLINE sized from the frame
- Audio: FIFO, sleep every period, PI locks
- UI: weight + slice; background BATCH / IDLE

Goal: **the right task at the right time** — on shared data that stays consistent.

End of Week 5

Youngjae Kim, Ph.D.

